# Probing Reinforcement-Learned Representations for Geo-Free Guard Placement in the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija[a,*], Jurica Maltar[a], Nathan Chappel, Domagoj Matijević[a]

[a] *School of Applied Mathematics and Informatics, University J. J. Strossmayer of Osijek, Croatia*

**Abstract**

Reinforcement learning can train models to solve hard combinatorial problems from raw inputs, but what such a model has learned, as opposed to what it outputs, is hard to establish. We study this on the vertex-guard Art Gallery Problem under a constraint we call *geo-free* inference: at test time the model sees only vertex coordinates, never a visibility computation, though training and evaluation use visibility freely. A pointer-network policy trained by a coverage-aware reward places guards economically but leaves a tail of under-covered polygons that grows beyond the training sizes. Freezing its encoder and reading the embeddings with a small classifier removes most of that tail at a cost in guards, cutting under-covered polygons several-fold in distribution and about ten-fold on instances five times the training size. Compared at equal guard count, the encoder rather than the classifier's own capacity accounts for the gain, and this holds on four independently trained policies; even a linear readout of the frozen features ranks guards above non-guards. The representation is not invariant, however: rotating or reflecting a polygon costs a large part of the gain. We conclude that the reinforcement-trained encoder holds much of the guard-placement structure its decoder failed to express, tied to the conventions of the instances it was trained on. The method measures what such a model has learned; it is not a competitive Art Gallery solver.

*Keywords:* Art Gallery Problem, neural combinatorial optimization, reinforcement learning, pointer networks, representation probing

## 1. Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [1, 2], with applications in

*Corresponding author

*Email addresses:* `dseverdi@mathos.hr` (Domagoj Ševerdija), `jmaltar@mathos.hr` (Jurica Maltar), `nathan.s.chappel@gmail.com` (Nathan Chappel), `domagoj@mathos.hr` (Domagoj Matijević)

surveillance, sensor placement, and robotics [3, 4]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon's vertices; the problem remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We refer to inference under this policy as *geo-free*: the network reads only vertex coordinates at the moment it places guards, never querying a visibility oracle (defined precisely in Section 2.3). We are not asking whether a learned solver can match a classical greedy heuristic on guard count; it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

*Why geo-free inference.* One could object that the coordinates are given, so a solver could just compute visibility from them; a visibility-based greedy does exactly that, at a lower guard count. Geo-free inference withholds the computation, not the information: the geo-free policy and classical greedy start from the same coordinates, but greedy evaluates visibility at every step while the geo-free model must have learned it. This makes the constraint a measurement tool rather than a handicap. Section 2.3 makes this precise: a solver allowed to query a visibility oracle at inference could simply simulate greedy or local search, confounding what it learned with what the oracle computes for it. Withholding the oracle is therefore a fair restriction — it denies the model a computation, not information.

Geo-free inference also shifts where the visibility cost is paid. In a classical pipeline that cost is the largest per-instance expense (a visibility polygon per vertex, then a polygon-union coverage per candidate set), and it recurs for every polygon. A geo-free policy pays it once, at training, and then answers each instance with a single coordinate-only forward pass in tens of milliseconds on a GPU, with model memory independent of $n$ (Sections 5.3 and 6.6). This trade of a high fixed cost for a low marginal one could matter where many instances must be solved and an approximate answer is acceptable, and that the trade is even available is not an assumption: the probing study is what shows the required visibility structure can be learned from coordinates. We deliberately do not rest the paper on that argument, however, because the conditions under which it pays are narrow. A geo-free pass returns a guard *set*, not a coverage value, so any use that needs to *score* coverage still calls the geometry the method was avoiding; and where a covering set is wanted as a starting point, classical greedy supplies a far better one at close to the optimum (Section 6.1) for a precompute cost that is negligible beside any exact downstream solve. Geo-free inference is therefore best understood here as a measurement instrument rather than a deployment recipe: it withholds a computation in order to isolate what the learner internalized from what an oracle would have supplied (Section 2.3). Where an exact oracle is cheap to query, a classical solver is simply the better tool.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [5] on Bradley–Terry preference pairs [6] drawn from the policy's own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [7, 8] with binary preferences between pairs of policy rollouts, preserving a usable gradient signal even when the reward saturates under the coverage constraint [5].

The policy works, to a degree. On a held-out in-distribution split its guard sets are competitive in cardinality with classical baselines, but the per-polygon coverage distribution has a tail: a non-trivial fraction of polygons end up below the 0.95 coverage gate one would want a deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, offers no coverage guarantee.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier called *probe* conditioned on the policy's frozen encoder embeddings (together with the vertex coordinates and a binary indicator of the policy's own guard set, all geo-free), and predict a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference, and it compresses the coverage tail substantially both in and out of distribution. A no-encoder ablation that zeroes the embeddings while keeping the coordinates and seed indicator (Section 6.2), together with a probe-capacity ladder that holds the encoder fixed while shrinking the probe, isolates the encoder's contribution from the other inputs and from the probe's own capacity. Those comparisons have to be made at *matched guard budget* rather than at a common threshold, because removing an input also changes how many vertices the probe selects and extra guards raise coverage on their own; at a fixed threshold the ordering can invert. Read that way, we take the gap as evidence that the reinforcement-trained encoder already carries guard-relevant structure a small probe can turn into a gate-clearing decision, so the policy's coverage tail reflects, on the better-supported reading, what its decoder expressed rather than a limit of the representation. The cost of clearing the tail is a controlled rise in guard count, which we report explicitly.

*Contributions.* This paper makes four contributions.

**C1** **A representation–decoder decomposition for neural combinatorial optimization.** We adapt the probing methodology of representation analysis [9, 10, 11] to a reinforcement-trained combinatorial solver: freezing the policy's encoder and training a single-shot per-vertex probe over its embeddings (with the vertex coordinates and the policy's seed) isolates what the learner *represents* from what its decoder *expresses*, with a $2 \times 2$ encoder–seed ablation and a probe-capacity ladder (linear, attention-free MLP, full probe) as controls. Concurrent work has begun probing neural *routing* solvers [12, 13]; to our knowledge ours is the first probing study of an RL-trained policy on a *constrained geometric covering* problem, and the first to read a coverage decision (per-vertex guard membership) from a frozen encoder under a geo-free inference constraint.

**C2 Out-of-distribution generalization at scale, replicated across policies.** On a large held-out set spanning the training size range and extending to roughly five times beyond it, the probe raises the average fraction clearing the 0.95 coverage gate across four probe seeds from 85% (policy seed) to 99%; the gain holds on the genuinely larger-than-training polygons alone (85% $\rightarrow$ 98.7%, Section 6.3). It is not a property of one training run: three further policies trained from scratch under the same recipe show the same tail-closing effect out of distribution, and the encoder's matched-budget advantage holds on all four in distribution (Section 6.4).

**C3 Geo-free inference as a protocol.** Restricting test-time inputs to vertex coordinates and learned features derived from them, never a visibility oracle, isolates what a learner internalizes from what such an oracle could compute for it, making "no explicit geometric knowledge" a measurable property rather than a rhetorical one (Section 2.3).

**C4 A structural finding on post-processing.** Learned iterative editors fail to improve the policy's seed without dropping coverage (error accumulation, stop-time miscalibration, and train/inference drift), whereas the single-shot probe avoids all three by design and is empirically stable under iteration: re-running it on its own output changes the prediction negligibly, drifting into a slow guard-shedding contraction only as the threshold is pushed toward the decision boundary. We measure this over $t \in [0.5, 0.8]$, above the reported operating range, because a higher threshold selects fewer vertices and so tests the property where it is most likely to fail (Section 6.5). We read this as support for the single-shot design as a stable reading of the encoder; the fixed point is an empirical property, not a proof that the probe is a clean representation readout.

We do not claim to beat classical greedy or local-search heuristics on guard count; we claim, and document, that vertex-guard placement is learnable to a measurable degree by a reinforcement method that never reads an explicit visibility object at inference. The components we use, pointer networks, Bradley–Terry preference optimization, frozen-representation probing, and a Transformer per-vertex classifier, are all established. Our contribution is their combination and adaptation to a constrained geometric covering problem, and the representation–decoder measurement it enables.

The remainder of the paper is organized as follows. Section 2 establishes notations and definitions. In Section 3 related work is reviewed. Section 4 describes the reinforcement method and the representation probe. Finally, Section 5 describes the experimental setup, Section 6 reports the results, followed by discussion in Section 7, limitations in Section 8, and the conclusion in Section 9.

## 2. Problem and Notation

*2.1. Polygons and visibility*

Let $P \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with $n$ vertices at coordinates $x_1, \ldots, x_n \in \mathbb{R}^2$ and vertex index set $V(P) = \{1, \ldots, n\}$. For two points $p, q \in P$ we say that $p$ *sees* $q$ when the closed segment between them stays inside the polygon, $\overline{pq} \subseteq P$. Guards are placed at vertices: for a vertex-guard set $S \subseteq V(P)$, the visibility region and area-normalized coverage are

$$\mathrm{Vis}_P(S) \;=\; \{\, q \in P : \exists\, i \in S,\ x_i \text{ sees } q \,\}, \qquad \mathrm{Cov}_P(S) \;=\; \frac{\mathrm{Area}(\mathrm{Vis}_P(S))}{\mathrm{Area}(P)} \in [0, 1]. \tag{1}$$

*2.2. The vertex-guard variant*

The *vertex-guard Art Gallery Problem (AGPVG)* asks for the smallest vertex-guard set that covers the whole polygon. We define the *vertex-guard optimum* directly:

$$\mathrm{OPT}(P) \;=\; \min\bigl\{|S| : S \subseteq V(P),\ \mathrm{Cov}_P(S) = 1\bigr\}. \tag{2}$$

Restricting guards to $V(P)$ reduces placement to a finite discrete choice over $n$ vertices, the natural action space for a pointer-network policy [14, 8, 15, 16], while retaining the full NP-hardness and non-uniform visibility structure of the problem.

Note that equation (2) is a *constrained* single-objective problem: minimize the guard count subject to the hard constraint $\mathrm{Cov}_P(S) = 1$ and is a discrete set-cover instance, where coverage is the submodular union of per-guard visibility regions. This is what separates it from the routing problems on which neural combinatorial solvers are usually studied, where any permutation is feasible and only a single scalar cost is minimized. A learner denied a visibility oracle at inference (the geo-free constraint, Section 2.3) cannot certify the coverage constraint while placing guards; it must instead trade coverage against guard count, so the single constrained objective becomes a two-axis operating curve.

*2.3. Geo-free inference*

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon's vertex coordinates and learned features derived from them: no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step, as classical local search and active-search decoding both do, can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes "no explicit geometric knowledge" a measurable property.

## 3. Related Work

This section places the present work in four lines of research: algorithms for the Art Gallery Problem (classical and learned), neural combinatorial optimization with reinforcement learning, supervised post-processing of learned solvers, and probing of learned representations.

*The Art Gallery Problem.* Lee and Lin [1] established NP-hardness for several variants of the AGP; O'Rourke [2] surveys the foundational geometric and combinatorial results. Work on the problem has traditionally followed three lines. Exact algorithms solve smaller instances to optimality, most effectively via integer programming: Couto et al. [17] compute exact vertex-guard optima this way, and we treat their published instance library [18], with its exact optima, as the source of polygons in our experiments. Approximation algorithms provide provable guarantees: Ghosh [19] gives a logarithmic-factor method for guarding simple polygons. Heuristics without theoretical guarantees, of which the classical greedy rule [20] that places guards to maximize marginal coverage is the most common, are reliable on small to mid-sized polygons. Because area coverage is a monotone submodular function of the guard set, this marginal-coverage rule is exactly the greedy submodular-maximization heuristic and inherits its diminishing-returns approximation guarantee [21]; the guarantee is available only to a solver that can evaluate marginal coverage while it selects, which a geo-free learner cannot (Section 2.3), so we forgo such guarantees by construction, the price of measuring what the representation has learned rather than what an oracle computes. The same exact/approximation/heuristic split recurs across the problem's many variants; for instance, guarding 1.5-dimensional terrains admits constant-factor approximations [22, 23]. Local-search refinement of a candidate guard set by swapping or removing vertices is the standard high-quality post-processing step. We use classical greedy purely as a non-learned evaluation reference, not in competition with the reinforcement method on guard count.

Reinforcement learning has been applied to the AGP and to closely related coverage problems before, but in settings quite different from ours. Liao et al. [24] discretize the polygon into a grid and train a per-instance tabular Q-learning agent that trades guard count against covered area; Kaba et al. [25] cast the 3D view-planning problem, a sensor-coverage set-cover, as an MDP solved with value-based RL (SARSA, Q-learning, temporal-difference) guided by a geometry-aware score, outperforming the greedy baseline. Both methods consult coverage or visibility while placing guards and learn a fresh policy for each instance. We differ on three axes: the action space is the polygon's own vertices rather than grid cells or candidate viewpoints; a single amortized policy generalizes to unseen polygons without retraining; and inference is geo-free, with no visibility computation at the moment guards are placed.

*Reinforcement learning for combinatorial optimization.* Pointer networks [14] introduced the architecture of attending over input tokens to produce variable-length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [8] using REINFORCE [7] with a learned baseline; Deudon et al. [26] similarly train TSP construction heuristics by policy gradient. Subsequent work introduced attention-only architectures for routing problems [15], policy-optimization variants such as POMO [16] that exploit problem symmetries, and Transformer pointer policies such as Pointerformer [27] aimed at generalizing to large instances — a concern we share, since our evaluation stresses out-of-distribution polygon sizes. We adopt the more recent preference-optimization (PO) recipe of Pan et al. [5], which trains the policy with a Bradley–Terry [6] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [28].

A recurring feature of this literature is worth making explicit, because AGP departs from it. Routing problems such as the TSP carry a single scalar objective and free feasibility (any permutation is a valid tour), and even constrained variants such as capacitated routing usually enforce feasibility by masking infeasible actions at decode time, which presumes an oracle for the constraint. Vertex-guard AGP has neither property: it is a constrained set-cover problem (Section 2.2), and under geo-free inference the coverage constraint cannot be masked in, so it can only be learned and traded off against guard count, the operating curve our threshold sweep traces. The closest learned analogues are RL methods that approximate Pareto frontiers for bi-objective routing [29, 30]; but those balance two *soft* costs, whereas one of our two axes is a hard feasibility constraint, making PA-Net's Lagrangian relaxation of a constraint [29] a closer match than a symmetric Pareto front. On the optimization side, Caramanis et al. [31] prove that policy-gradient solution-samplers enjoy benign landscapes for a broad class of problems (Max-/Min-Cut, Max-$k$-CSP, bipartite matching, TSP), but their guarantee assumes a single cost that is *bilinear* in instance and solution features and imposes no hard feasibility constraint. AGP's guard-count term is linear and would fit, but its coverage term is the submodular union of visibility regions and its full-coverage requirement is a hard, geo-free-uncertifiable constraint, placing AGP outside their analyzed class and helping explain why a naive policy gradient struggles here (Section 4.1).

*Supervised post-processing of learned solvers.* An orthogonal line of work post-processes a base solver's output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [32] and its variants partially address the latter. We use a single-shot supervised post-processor (the SetPredictor) not as a competing solver but as a *probe* of the RL-trained encoder's representation. The framing is methodological: if a small classifier reading frozen RL embeddings can recover the coverage the decoder left short, the RL encoder must already carry the relevant geometric information. Directly cloning search solutions with a recurrent decoder, by contrast, fails on this problem for a structural reason, the teacher-forcing cascade we return to in Section 6.5, which is why we read the frozen encoder with a single-shot, recurrence-free probe rather than a sequential supervised decoder.

*Probing learned representations.* The methodology of freezing a trained network and reading its internal representations with a small supervised classifier originates in the analysis of vision and language models: linear classifier probes diagnose what intermediate layers encode [9], control tasks calibrate how much of a probe's accuracy reflects the representation rather than the probe's own capacity [10], and Belinkov [11] surveys the family. A central methodological caveat runs through this literature: a sufficiently expressive probe can achieve high accuracy by *computing* the target itself rather than reading it off the representation, so probing results must be interpreted against a capacity control [10]. We take this concern seriously below by reporting a capacity ladder (a zero-hidden-layer linear probe, an attention-free MLP, and the full SetPredictor) alongside a no-encoder control task, so that the effect we attribute to the representation is the part that survives holding probe capacity fixed and removing the encoder.

Probing has only recently reached neural combinatorial optimization, and the closest work is concurrent. Zhang et al. [12] freeze trained routing models (AM, POMO, LEHD) on the TSP and CVRP and read their embeddings with *linear* probes for a low-level geometric quantity (pairwise Euclidean distance), for solution-structure signals (whether a link avoids a myopic greedy choice; whether two nodes lie on the same route), and for a capacity-constraint signal, with a coefficient-significance tool (CS-Probing) that localizes which embedding dimensions carry each. Narad et al. [13] similarly probe TSP embeddings for node-removal and edge-forbid sensitivity. We use the same freeze-and-probe method but probe something different in a different setting. Our problem is covering, not routing: under geo-free inference the coverage constraint cannot be enforced by decode-time masking, so the model must learn it and trade it against guard count. Our target is global, per-vertex membership in a guard set whose visibility regions must cover the polygon, rather than a pairwise quantity like inter-node distance; visibility depends on the whole boundary, so recovering it from the embedding says more about learned geometry than recovering a distance does. The geo-free limit on what the probe may read at inference has no counterpart in the routing studies. And where they probe linearly, avoiding the capacity question but seeing only linear signal, our probe is an expressive Transformer that also serves as the post-processor, so we run a capacity ladder (linear, attention-free MLP, full Transformer; Section 6.2) to confirm the signal comes from the representation and not the probe. The learning frameworks differ only in recipe, our Bradley–Terry preference optimization versus their policy-gradient (AM, POMO) or supervised (LEHD) training, but since AM and POMO are also RL-trained, this is a difference in algorithm, not in paradigm, and we do not rely on it to distinguish our work. Our no-encoder ablation is the control task [10]: it holds probe capacity and training fixed while removing the representation, so the gap is attributable to the encoder.

## 4. Method

We describe the reinforcement method (Section 4.1) and the single-shot representation probe (Section 4.2). The empirical diagnostic that most motivates the single-shot design, i.e., why iterative editors fail to improve the seed, is deferred to the results (Section 6.5).

### *4.1. The reinforcement method: PO/BT pointer policy*

*Architecture.* The policy is a compact pointer network with a single-layer, unidirectional LSTM [33] encoder and an attention-based decoder. A polygon $P$ with vertex coordinates $(x_1, \ldots, x_n)$ is consumed coordinate-wise (no visibility input). Running the LSTM over the vertex sequence yields per-vertex hidden states that serve as the embeddings $(h_1, \ldots, h_n)$, $h_i \in \mathbb{R}^d$, where $d$ is the encoder embedding dimension (its value, and those of all dimensions introduced below, is set in Section 5.2). The decoder attends over the encoder at each step, emits a vertex index or an *end-of-sequence* (EOS) token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \ldots, \pi_{|\pi|})$ while the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$. All metrics, i.e., coverage, $|S|/n$, $|S|/\text{OPT}$, are computed from $S(\pi)$. We denote the joint encoder–decoder distribution by $p_\theta(\pi \mid P)$, where $\theta$ collects all policy parameters.

The encoder has no training signal of its own: its weights change only because the reward gradient back-propagates from the decoder's rollouts. If the encoder already holds the geometric information needed for feasibility, a small classifier reading its frozen embeddings should be able to recover the guard set the decoder failed to express. This is why we probe the encoder inside the trained policy by freezing it and attaching a small classifier. By contrast, a standalone model, i.e., an encoder + a probe trained from scratch, would reflect its own learning, not what the reinforcement policy internalized. The policy's greedy decode additionally supplies the seed guard set that the representation probe (Section 4.2) later refines.

*Why an autoregressive pointer.* Vertex-guard placement is a set-cover problem: whether a vertex should be a guard depends on what the guards already chosen leave uncovered. An autoregressive pointer captures this dependency naturally by conditioning each pick on the partial solution (Eq. (5)), and it decides cardinality through the EOS token rather than a fixed threshold. A model scoring vertices independently would miss this structure — which is also why the probe (Section 4.2) is not a standalone solver: unable to be autoregressive, it takes the policy's greedy seed as an input instead.

The reason the decoder is necessary once the probe works, i.e., why we cannot discard it, is that the encoder's representation was shaped entirely by the reward

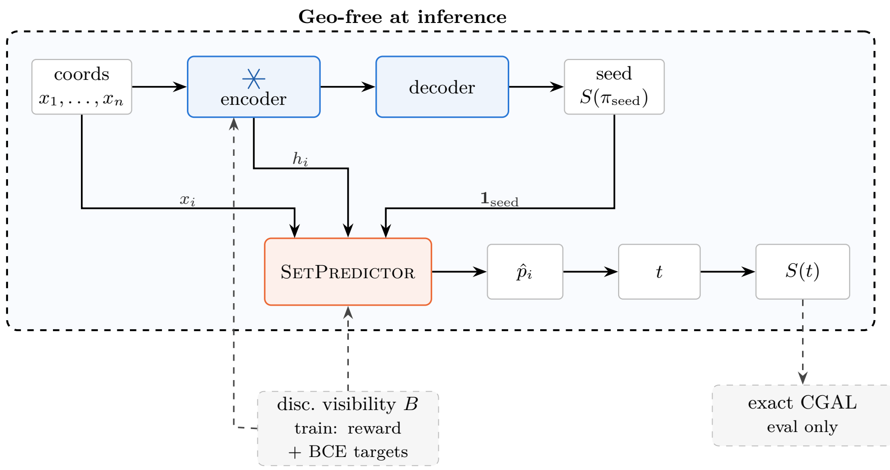


Figure 1: The full inference pipeline. The frozen LSTM encoder (trained by PO/BT, Section 4.1) turns vertex coordinates into per-vertex embeddings $h_i$; its own decoder greedily decodes a seed guard set $S(\pi_{\text{seed}})$. The SetPredictor (Section 4.2) reads the frozen embeddings, the coordinates, and the seed indicator (Eq. (7)) and outputs a per-vertex probability $\hat{p}_i$, thresholded at $t$ into the final guard set $S(t)$. Everything inside the dashed boundary is geo-free: no visibility object is read at the moment guards are placed (Section 2.3). Discretized visibility $B$ (Eq. (4)) shapes the PO/BT reward and the SetPredictor's BCE targets $S^\star_{\text{LS}}$ during training only; exact CGAL scores both guard sets only at evaluation (Section 5.3) — both cross the geo-free boundary from outside it, never from within.

gradient flowing through the decoder, and the probe reads the decoder's seed as a feature. Removing the decoder removes both the representation and the probe's input. What we compare is encoder-and-decoder against encoder-and-probe, not the decoder against no-decoder. Reading the frozen encoder with a probe also recovers coverage on polygons far larger than those seen in training (Section 6.3), which autoregressive pointer policies alone cannot do [14, 27].

*Reward and rollouts.* For each polygon $P$ we sample $R$ rollouts from the policy and score each rollout's guard set $S(\pi)$ by a scalar utility that gates coverage at $\tau$ and then prefers smaller guard sets,

$$u(\pi \mid P) = \min\bigl(\mathrm{Cov}_P(S(\pi)),\, \tau\bigr) - \lambda \frac{|S(\pi)|}{|V(P)|} - \rho \max\bigl(0,\, \tau - \mathrm{Cov}_P(S(\pi))\bigr), \tag{3}$$

where $\tau = 0.99$ is the coverage gate, $\lambda = 1.0$ weights guard usage, and $\rho = 3.0$ penalizes coverage below the gate (Section 5.2). Capping the coverage term at $\tau$ removes any incentive to over-cover, so once a rollout meets the coverage gate the utility is driven purely by cardinality. Coverage during training is computed with the discretized-visibility sampler described next ($M = 500$ interior points per polygon), a cheap approximation of the exact coverage area.

*Discretized-visibility coverage.* Computing the exact coverage of a guard set (the area of the union of its vertices' visibility polygons) is too costly to repeat for the many rollouts and local-search candidates scored per polygon, so during training we estimate it by Monte-Carlo sampling. For each polygon we draw $M = 500$ points uniformly from its interior (rejection sampling within the bounding box, under a fixed per-polygon seed) and, for each vertex $i$, compute its *exact* visibility polygon $\mathrm{Vis}_P(\{i\})$ once with CGAL triangular-expansion visibility [34]. Recording which samples fall inside each visibility polygon yields a boolean visibility matrix $B \in \{0,1\}^{|V(P)|\times M}$ with $B_{is} = 1$ iff sample $s$ is visible from vertex $i$, and the coverage of a guard set $S$ is estimated by the covered-sample fraction

$$\overline{\mathrm{Cov}_P(S)} \;=\; \frac{1}{M}\left|\left\{\, s : \exists\, i \in S,\; B_{is} = 1 \,\right\}\right|, \tag{4}$$

a uniform Monte-Carlo estimate of the area ratio $\mathrm{Cov}_P(S)$ of Eq. (1), evaluated as a bitwise OR over the rows of $B$ indexed by $S$. Because $B$ depends only on the polygon, it is computed once and cached, so every subsequent coverage query during a rollout or a local-search edit costs an $O(|S|\,M)$ bitwise operation rather than a polygon-union computation. The per-vertex visibility polygons are exact; the only approximation is replacing the area integral by a sample average — each sample is an independent Bernoulli trial with success probability $\mathrm{Cov}_P(S)$, so the estimate is a binomial proportion whose standard error is the familiar $\sqrt{\mathrm{Cov}_P(S)(1-\mathrm{Cov}_P(S))/M} = O(1/\sqrt{M})$. Exact coverage, ie the area of the union, computed exactly from the polygon geometry rather than sampled is reserved for evaluation (Section 5.3).

*Preference-optimization loss.* We score a rollout by its *length-normalized* log-likelihood — the mean per-token log-probability under the factorization

$$p_\theta(\pi \mid P) = \prod_t p_\theta(\pi_t \mid \pi_{<t}, P)$$

,

$$\bar{\ell}_\theta(\pi \mid P) \;=\; \frac{1}{|\pi|}\sum_{t=1}^{|\pi|} \log p_\theta(\pi_t \mid \pi_{<t}, P), \tag{5}$$

which keeps the preference from being biased by guard-set size, since AGP solutions have variable length (cf. Pan et al. [5]). For each pair $(\pi^+, \pi^-)$ of rollouts with $u(\pi^+ \mid P) > u(\pi^- \mid P)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\mathrm{BT}}(\theta) \;=\; -\mathbb{E}_{(P,\pi^+,\pi^-)}\left[\,\log\sigma\big(\alpha\,[\,\bar{\ell}_\theta(\pi^+ \mid P) - \bar{\ell}_\theta(\pi^- \mid P)\,]\big)\,\right], \tag{6}$$

where $\alpha > 0$ is a temperature scaling the preference margin ($\alpha = 0.05$; Section 5.2). The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

The choice of PO over REINFORCE is deliberate. Once a rollout meets the coverage constraint, the reward of Eq. (3) flattens and the policy-gradient signal on guard count vanishes — a saturation effect specific to hard coverage constraints, not covered by benign-landscape guarantees for unconstrained

| Training objective | Mean cov | Mean $|S|$/OPT |
|---|---|---|
| REINFORCE, learned-critic baseline | 0.904 | 3.49 |
| REINFORCE, greedy rollout baseline | 1.000 | 7.00 |
| PO / Bradley–Terry (ours) | **0.978** | **1.30** |

Table 1: Training-objective comparison on the combined `dev+test` pool (greedy decode). Coverage is geometric polygon coverage; $|S|$/OPT is the mean approximation ratio. The two REINFORCE rows are short 30-epoch runs scored under an *earlier* evaluation protocol: computed on the combined `dev+test` pool *before* the train/eval-leakage deduplication that produced the clean splits used in all other tables (Section 5.1), and with far fewer training epochs than the 200-epoch PO/BT policy. They are reported only as evidence for the *choice of training objective* (REINFORCE over-guards by 3.5–7× the optimum, a gap far too large to be a protocol artifact), and their absolute numbers are *not* matched cell-for-cell to the headline tables. The PO/BT entry is shown under the same earlier protocol for reference (which is why its $|S|/\mathrm{OPT} = 1.30$ here differs from the clean-split `test` value in Table 3); the policy it refers to is the one used throughout the paper.

problems [31]. We observe this failure directly: a value-critic REINFORCE policy over-guards by roughly 3.5× the optimum and a greedy-baseline variant by about 7× (Table 1). Bradley–Terry preferences between rollout pairs remain informative even when both rollouts saturate, since the signal depends on which rollout is relatively better, not on the absolute reward level. PO/BT reaches comparable coverage at 1.3× the optimum. At the end of training the encoder is frozen and treated as a fixed feature extractor for the rest of the paper; Figure 2 shows the late-phase policy holding a stable operating point: good coverage at low guard cost rather than collapsing onto a single axis.

### *4.2. Probing the representation: a single-shot per-vertex classifier*

The policy learns to decrease the cardinality but leaves a coverage tail (Section 6.1). We first verified that this tail is *reachable*: running a local-search editor (add/remove/swap) on the policy's own greedy seed lifts every below-gate in-distribution polygon back over the 0.95 gate while *reducing* the guard count (the *LS on policy seed* row of Table 3). We treat this as a sanity check on *reachability*, not as a diagnosis of cause: it shows the missing coverage is a handful of local edits away from the seed, which is consistent with a decoder that stopped early but does not by itself establish that against the alternative of a representation that is merely incomplete. Distinguishing those is what the encoder ablation is for (Section 6.2). As a by-product the editor yields a concrete target set $S^{\star}_{\mathrm{LS}}$. The open question is then whether that refinement can be *learned* from the frozen encoder, geo-free. A learned *iterative* editor over the seed cannot: it does not close the tail in any coverage-preserving regime (Section 6.5), because error accumulation, stop-time miscalibration, and train/inference drift make a rollout-based post-processor unreliable here, contrary to fine-tuning arguments of the policy in [5]. We therefore probe what the policy has learned with a *single-shot, rollout-free* classifier that recovers the coverage decision the decoder failed to make. Its inputs are geo-free (the frozen encoder embedding, the vertex coordinates, and the policy's seed indicator), with no visibility object

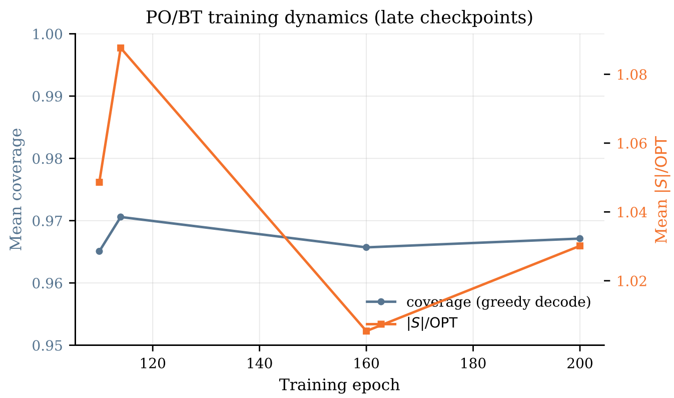


Figure 2: PO/BT training dynamics over the four late checkpoints (epochs 110, 114, 160, 200), evaluated on a fixed held-out sample of 100 polygons; per-epoch logging was not preserved, so early training is not shown (Section 8). Mean per-polygon coverage (left axis, zoomed to $[0.95, 1.0]$) and $|S|/\mathrm{OPT}$ (right axis) of the greedy-decoded policy stay within a narrow band with no sign of collapse; the movement is non-monotone rather than a clean trend, which is all four checkpoints can establish.

at inference. Having no rollout and no stop decision, a single pass has nothing to drift and nothing to miscalibrate. We call it the SetPredictor and present it primarily as a representation diagnostic; the threshold sweep in Section 6 makes the diagnostic concrete, but the operating-curve framing is a side-effect of the measurement, not a deployment recipe.

For polygon $P$ with vertex coordinates $x_1, \ldots, x_n$, the input feature for vertex $i$ is

$$z_i = [\, h_i \,;\, x_i \,;\, \mathbf{1}\{i \in S(\pi_{\text{seed}})\} \,] \in \mathbb{R}^{d+3}, \tag{7}$$

that is, the *frozen* $d$-dimensional encoder embedding $h_i$ from the PO/BT policy, the 2D coordinates $x_i$, and a single binary indicator for whether vertex $i$ is selected by the policy's greedy-decoded seed $\pi_{\text{seed}}$.

The architecture is a small Transformer encoder (parameters $\phi$) over the $n$ vertex tokens with a sigmoid output head per token (sizes given in Section 5.2). The features $z_i$ are linearly projected to width $d$ and passed through $L$ pre-norm self-attention blocks (each with $H$-head self-attention at width $d$, a GELU feed-forward sublayer of expansion factor $f$, and residual connections), followed by a final layer normalization; we write $v_i$ for the resulting per-vertex hidden state. This $v_i$ is the SetPredictor's *own* encoding of vertex $i$, and is distinct from the frozen policy embedding $h_i$, which enters only as part of the input feature $z_i$ in Eq. (7). From the $(v_1, \ldots, v_n)$ we build two context vectors by mean-pooling: a *seed* context $c_{\text{seed}} = \frac{1}{|S(\pi_{\text{seed}})|} \sum_{i \in S(\pi_{\text{seed}})} v_i$, averaged over the vertices the policy placed in its seed, and a *global* context $c_{\text{glob}} = \frac{1}{n} \sum_{i=1}^{n} v_i$, averaged over all vertices of the polygon. Both are single vectors shared across vertices. Each per-vertex prediction concatenates the vertex's own $v_i$ with these two context vectors and passes the result through a 2-layer GELU MLP head ($3d \to d \to 1$):

$$\hat{p}_i \;=\; \sigma\!\big(\mathrm{MLP}([\, v_i \,\|\, c_{\mathrm{seed}} \,\|\, c_{\mathrm{glob}} \,])\big), \qquad i = 1, \ldots, n. \tag{8}$$

The output $\hat{p}_i \in [0, 1]$ is interpreted as the probability that vertex $i$ should be in the final guard set. Thresholding at $t \in (0, 1)$ yields the guard set

$$S(t) \;=\; \{\, i \in \{1, \ldots, n\} : \hat{p}_i \geq t \,\}. \tag{9}$$

*Objective.* The probe is trained by supervised learning to predict membership in an LS-improved target set $S^{\star}_{\mathrm{LS}}(P)$ (the policy's seed refined by local search, constructed as described in Section 5.2), so it is not itself a reinforcement learner. We supervise on this refinement rather than on the exact optimum for two reasons: guard membership at the optimum is non-unique, since the constraint in Eq. (2) fixes cardinality, not which vertices are chosen (Section 6.5), so an optimal set is an ill-posed per-vertex label, and the optimum is in any case unavailable at scale (Section 5.3); the LS endpoint, by contrast, is a well-defined, coverage-restoring refinement of the policy's *own* seed. With $y_i = \mathbf{1}\{i \in S^{\star}_{\mathrm{LS}}(P)\}$, a positive-class weight $w_+$ that balances the empirical non-guard-to-guard ratio, and $\mathcal{B}$ a mini-batch of polygons, the per-vertex weighted binary cross-entropy is

$$\mathcal{L}_{\mathrm{BCE}}(\phi) \;=\; -\frac{1}{\sum_{P \in \mathcal{B}} |V(P)|} \sum_{P \in \mathcal{B}} \sum_{i=1}^{|V(P)|} \big[\, w_+ y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \,\big]. \tag{10}$$

Whether closing that tail is the encoder's doing rather than the probe's own capacity is settled not here but by the controls of Section 6.2: the no-encoder ablation and a zero-capacity linear probe.

*Inference and the threshold knob.* At inference we run the policy encoder once on the coordinates, greedy-decode the seed $\pi_{\mathrm{seed}}$, form the features $z_i$ of Eq. (7), run one SetPredictor forward pass to obtain $(\hat{p}_1, \ldots, \hat{p}_n)$, and threshold to return $S(t)$ (Eq. (9)). The policy seed, the local-search editor, and the discretized-visibility scoring are all *training-time* scaffolding used only to build the labels $S^{\star}_{\mathrm{LS}}$; at inference the method is this single geo-free forward pass — no seed-refinement loop, no local search, no visibility object, so LS is part of how the probe is *trained*. The threshold $t$ is the single knob trading coverage for guard count. We report the headline results at $t \in \{0.20, 0.25, 0.30\}$ (Section 6.2) and sweep it over the full range $[0.05, 0.80]$ where the comparison requires it: to trace the whole operating curve (Table 8) and to place ablation arms at matched guard budget rather than at a common threshold (Table 6). Re-running the probe with its own output as the updated seed indicator changes the prediction negligibly (Section 6.5), so we use a single pass throughout.

## 5. Experiments

### *5.1. Dataset and partition*

We draw polygons primarily from the *Art Gallery Problem with Vertex Guards* (AGPVG) instance library of Couto, de Rezende, and de Souza [18,

| Split | # polygons | $n$ range | Use |
|---|---|---|---|
| train | 8,867 | 8–198 | Pointer + SetPredictor training |
| dev | 840 | 16–198 | SetPredictor checkpoint selection |
| test | 362 | 8–192 | Held-out evaluation (in-distribution) |
| ood | 2,081 | 8–1,000 | Out-of-distribution evaluation |
| ood-large | 285 | 600–2,250 | Extreme-OOD evaluation |

Table 2: Dataset partition. The `dev` and `test` splits share the same source library files, partitioned 70/30 by seeded random shuffle (seed 1234): `dev` is used for checkpoint selection and `test` is the in-distribution held-out evaluation. The `ood` split is a larger held-out set spanning the training size range and extending well beyond it, to $n = 1000$ (about five times the training maximum); its genuinely larger-than-training polygons ($n > 198$) are analyzed separately in Section 6.3. The `ood-large` split is used for an extreme-OOD evaluation (Table 10).

17], which collects several polygon families — random orthogonal (including large-area `fat` and small-area `min` variants), random simple (`randsimple`), random von Koch (`randvon`), complete von Koch (`vonkoch`), and boundary-simple (`simple`) — over vertex counts $n$ ranging from 8 to 2,500. The library distributes vertex-guard optimal solutions for its instances, which we use directly as OPT.

We supplement the library with 313 additional random simple polygons (`random` class, $n \in \{20, 40, \ldots, 600\}$, 30 instances per size) to increase training density for random simple polygons in the in-distribution range; these share the same rational-coordinate format, and their vertex-guard optima were computed by solving the vertex-guard set-cover integer program. The splits we work with are listed in Table 2.[1] The held-out evaluation splits are deduplicated against `train` to remove any instance shared with it (train/eval leakage); every evaluation number in this paper is computed on these leak-free splits, and it is this deduplication that separates them from the earlier protocol under which the REINFORCE rows of Table 1 were scored.

### *5.2. Training setup*

The PO/BT policy is the released `lstm_bt` checkpoint: a *single* PO/BT training run (seed 1234, 200 epochs), from which we keep the checkpoint with the best greedy-decoded `train` performance. It uses $R = 8$ rollouts per polygon, Bradley–Terry temperature $\alpha = 0.05$, and reward weights $\lambda = 1.0$ and $\rho = 3.0$ at the gate $\tau = 0.99$ (Eq. (3)); all PO coverage rewards are computed from discrete-visibility sampling ($M = 500$), after which the encoder is frozen. During development we compared several training *recipes*, namely REINFORCE baselines (Table 1) and alternative fine-tuning objectives, and selected PO/BT among them by `train`-set performance. Several runs of the chosen recipe were trained, and the released policy is the one with the best greedy-decoded `train` performance among them; within a run, the retained checkpoint is likewise the

[1]The 70/30 `dev/test` partition uses a seeded random shuffle (seed 1234) rather than an alphabetical-name split due to instance naming

best on `train`, scored by coverage *minus* the selected vertex fraction. That penalty matters: coverage alone is maximized by over-guarding, and on one replication seed a coverage-only rule selects a near-degenerate early epoch (Section 8). Both selections use only the `train` split, so `test` and the OOD splits play no part in them.

*Inputs:* a polygon's vertices are consumed by the LSTM in the source-library file order, with no canonical start vertex and no orientation (clockwise/counter-clockwise) imposed, and the coordinates are min–max normalized per polygon to $[0,1]^2$ before the encoder. The normalization makes the policy invariant to translation and to axis-aligned rescaling by construction; it is *not* invariant to rotation, to reflection, or to a cyclic re-indexing of the vertices, and we find empirically that these symmetries are not acquired during training either. Of the three, only the vertex-order dependence admits a cost-free fix, by canonicalizing the traversal at inference; we quantify all of this in Section 8.

*Targets:* for each training polygon we obtain $S^{\star}_{\mathrm{LS}}(P)$ by running local search on the policy's seed under a coverage gate $\tau = 0.99$ against a discrete-visibility reference (Section 4.1); exact coverage-area computation is reserved for evaluation. These targets are *not* produced by a policy-independent classical solver: they come from a best-improvement editor (add/remove/swap) initialized at the policy's own greedy-decoded seed and scored on discretized visibility. We refer to this policy-seeded refinement as "LS" throughout; it is the same edit family whose *learned* counterpart we analyze in Section 6.5, here run to its best-improvement optimum as an oracle teacher rather than learned.

*Sizes:* the model dimensions of Section 4.2 take the following values:

| | |
|---|---|
| working width $d$ | 128 |
| self-attention blocks $L$ | 3 |
| attention heads $H$ | 8 |
| FFN expansion factor $f$ | 2 |
| policy parameters | $\approx$ 364k |
| SetPredictor parameters | 464,001 |

*Optimization:* weighted BCE (Eq. (10)) with positive-class weight $w_+ \approx 5.66$ (the empirical non-guard-to-guard ratio on `train`), 60 epochs, batch size 32, learning rate $3 \times 10^{-4}$, AdamW [35]. Every SetPredictor variant — the headline probe, the no-encoder ablation, and the capacity-ladder rungs (Section 6.2) is four independent runs with probe seeds $\{1234, 11, 22, 33\}$ (reported as mean $\pm$ std); all figures display seed 1234. These four seeds are always SetPredictor *probe* seeds; the underlying PO/BT policy is the single run described above, not re-seeded. Probe checkpoints and the decision threshold are selected on `dev`, whereas `test`, `ood`, and `ood-large` are touched only after training and threshold selection are complete. The headline threshold $t = 0.20$ is the coverage-favoring end of the swept range $\{0.20, 0.25, 0.30\}$, selected on `dev` before any `test` or OOD use. The 0.95 coverage gate does not by itself single it out, since every threshold in the sweep clears that gate on all `dev` polygons; what selects $t = 0.20$ is the stricter criterion we report alongside it, namely the count remaining below the 0.99 gate and the worst-case coverage $\min_P \mathrm{Cov}_P(S)$, on both of which it is the best of the three at the cost of the highest guard count. We report all three thresholds rather than tune $t$ per split, and the wider curve in Section 6.2 shows what the choice costs.

### *5.3. Evaluation protocol and metrics*

For each polygon we greedy-decode the policy's EOS-truncated seed and evaluate it with CGAL exact polygon visibility [34]; separately, we run one SetPredictor pass, threshold at $t$ to obtain $S(t)$, and evaluate $S(t)$ with CGAL. Let $N$ be the partition size. We report mean coverage $\frac{1}{N}\sum \mathrm{Cov}_P(S)$, the normalized guard count $|S|/n$, and the approximation ratio $|S|/\mathrm{OPT}$ against the vertex-guard optimum. Because the mean hides the tail, we also report the per-polygon counts $\#\{\mathrm{Cov}_P(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$ and the worst case $\min_P \mathrm{Cov}_P(S)$. We distinguish two notions throughout. *Exact feasibility* is full coverage, $\mathrm{Cov}_P(S) = 1$, as the problem defines it (Eq. (2)); we reserve the words *feasible* and *feasibility* for this exact sense. Separately, the counts $\#\{\mathrm{Cov}_P(S) \geq c\}$ report a family of *coverage gates*; we single out $c = 0.95$ as the 0.95 *coverage gate*, a near-feasibility reporting threshold, and report the gate-clearing rate $\#\{\mathrm{Cov}_P(S) \geq 0.95\}/N$ with Wilson 95% confidence intervals. Clearing the 0.95 gate does *not* mean a polygon is feasible in the exact sense. These reporting gates are distinct from the training-time coverage gate $\tau = 0.99$ of Eq. (3), which shapes the reward rather than scoring the output. Where we compare the policy's and the probe's gate-clearing rates on the same polygons, we test the paired per-polygon outcomes with an exact McNemar test. As noted in Section 5.1, OPT values are taken from the vertex-guard optimal solutions distributed with the AGPVG library (for library instances) or computed via ILP (for the `random` class). For 79 of the `ood-large` polygons (all with $n \geq 800$) no optimum is available — most are library instances whose distributed solutions do not cover them (they are open at the source), and for the remainder our ILP did not terminate within budget. Restricting $|S|/\mathrm{OPT}$ to the 206 solved instances would bias the ratio toward the easier 600–700-vertex polygons, so on `ood-large` we report coverage and the normalized guard count $|S|/n$ over all 285 polygons but no approximation ratio.

*Computational cost.* All experiments were run on a single workstation with an AMD Ryzen 9 3900X CPU, 64 GB RAM, and an NVIDIA GeForce RTX 2080 SUPER GPU (8 GB VRAM). Training the SetPredictor probe is light: about 5 s per epoch, $\approx$ 5 minutes for a 60-epoch run on the GPU above (the four-seed ensemble, $\approx$ 20 minutes); the frozen PO/BT policy is reused rather than retrained. The geometric preprocessing (the exact per-vertex CGAL visibility polygons and the $M = 500$ discretized-visibility matrix) is computed once per polygon and cached for the $\sim$9.9k library polygons, so it is not repeated across rollouts, edits, or seeds. At inference the method is a single geo-free forward pass, tens of milliseconds on the GPU above; Section 6.6 times it in full and sets it against the classical visibility-based pipeline. Exact CGAL coverage is used solely for evaluation and is the dominant offline cost, scaling with the guard-set size.

## 6. Results

The results follow the hypothesis: the policy places guards but leaves a tail (Section 6.1), reading the frozen encoder shortens it substantially (Section 6.2), the effect generalizes out of distribution (Section 6.3) and across independently trained policies (Section 6.4), and a single pass suffices because the probe is near-stationary under iteration (Section 6.5).

### *6.1. The policy places guards*

The geo-free policy on its own (Table 3, *seed* row) places guards at near-classical cardinality and clears the 0.95 coverage gate on 293 of the 362 `test` polygons (Wilson 95% CI in the table). The LS oracle (bottom row) clears the 0.95 gate on all 362 at lower cardinality still, but is not geo-free: it reads discretized visibility at every add/remove/swap step and serves here only as the probe's training target, not as a geo-free competitor. The results show that vertex-guard placement is learnable, to a degree, by a method that never reads an explicit visibility object at inference. The remaining 69 polygons fall below the 0.95 gate under the policy alone; this tail is the paper's central motivating observation (the distribution is shown in Section 6.2), and the method gives no coverage guarantee on its own. Figure 3 shows the seed, the seed plus probe at $t = 0.20$, and the optimum side by side for an in-distribution and an out-of-distribution polygon.

### *6.2. Reading the encoder shortens the tail*

The policy's mean coverage hides a bimodal distribution (Table 4): it reaches full coverage on only a handful of polygons and trails a long left tail, while the SetPredictor at $t = 0.20$ shortens that tail sharply and lifts the bulk above 0.99 (302 of 362 polygons against the policy's 97). The improvement is a genuine shift rather than a trade: on the displayed seed the probe lifts 57 of the policy's 69 below-gate polygons back over 0.95 while pushing 5 previously-clearing polygons below it, a net $69 \to 17$ (paired exact McNemar test on those 62 discordant polygons, $p \approx 3 \times 10^{-12}$); the same paired test is applied at OOD scale in Section 6.3. The shift also holds under the strict full-coverage requirement that defines feasibility ($\mathrm{Cov}_P(S) = 1$), where the probe covers 61 polygons exactly against the policy's 3 (Table 4), so the gain is not an artifact of reading only the loose 0.95 gate — though exact coverage remains the minority case, which is the limitation Section 8 returns to.

The next question we ask is whether closing that tail is the encoder's doing or coordinates alone suffice. We isolate the encoder's contribution by retraining the probe with the frozen embedding $h_i$ masked to zero on every forward pass, holding architecture, parameter count, optimizer, and training schedule identical (Section 5.2). Comparing the two probes at a common threshold is

| Method | Mean cov | #{Cov ≥ 0.95}/$N$ | Mean $\lvert S\rvert/n$ | Mean $\lvert S\rvert$/OPT |
|---|---|---|---|---|
| *Classical baseline (non-learned)* | | | | |
| Greedy | 0.9994 | 362/362 | 0.1524 | 1.0092 |
| *Learned policy / probe* | | | | |
| Pretrained pointer (seed) | 0.9689 | 293/362 (0.766,0.847) | 0.1664 | 1.0885 |
| SetPredictor full ($t = 0.20$) | **0.9912** ± 0.0024 | 349.8 ± 3.5/362 | 0.2475 ± 0.0050 | 1.6208 ± 0.0299 |
| SetPredictor, no-encoder ($t = 0.20$) | 0.9976 ± 0.0004 | 361.5 ± 0.5/362 | 0.4517 ± 0.0432 | 2.9405 ± 0.2654 |
| *Oracle editor (SetPredictor target, policy-seeded)* | | | | |
| LS on policy seed | 0.9948 | 362/362 | 0.1369 | 0.8854† |

Table 3: Held-out evaluation on `test` (362 polygons). Classical greedy is the only non-learned anchor. The *seed* row is the PO/BT policy decoded greedily; *LS on policy seed* is its local-search refinement — the SetPredictor's supervised target, not a classical baseline (Section 5.2). SetPredictor *full* and *no-encoder* are the headline probe and its zeroed-embedding ablation, each mean ± std over four seeds $\{1234, 11, 22, 33\}$ at $t = 0.20$ (Section 6.2). At this common threshold the two probes sit at different operating points — the no-encoder ablation clears the 0.95 gate on every polygon but spends 1.8× the guards ($|S|$/OPT 2.94 against 1.62) — so the gate counts here should not be read as an encoder comparison; Section 6.2 makes that comparison at matched guard budget instead. Wilson 95% CI shown only where the proportion is informative.
†$|S|$/OPT $< 1$ for the oracle editor because it halts at a disc-vis coverage gate ($\tau = 0.99$) and slightly undercovers (mean exact coverage 0.995), so it uses fewer guards than the *full-coverage* optimum. This is an artifact of the training reward, not a loose OPT; the $|S|/n$ column is free of this caveat.

| Method | #{Cov = 1} | #{Cov ≥ 0.999} | #{Cov ≥ 0.99} | #{Cov ≥ 0.95} | min Cov |
|---|---|---|---|---|---|
| Pretrained pointer (seed) | 3 | 15 | 97 | 293/362 | 0.830 |
| SetPredictor $t = 0.20$ (full) | 61 | 128 | 302 | 345/362 | 0.857 |
| SetPredictor $t = 0.20$, no-encoder | 151 | 206 | 330 | 362/362 | 0.951 |
| SetPredictor $t = 0.25$ (full) | 54 | 115 | 296 | 343/362 | 0.856 |
| SetPredictor $t = 0.30$ (full) | 43 | 102 | 288 | 343/362 | 0.848 |

Table 4: Per-polygon coverage distribution on `test` (362 polygons). Counts are polygons reaching each coverage threshold; *min* $\mathrm{Cov}_P(S)$ is the worst polygon. The policy has a long left tail; the SetPredictor shifts the whole distribution right. At the common threshold $t = 0.20$ shown here the no-encoder ablation reaches the stricter gates more often than the full probe (330 against 302 at ≥ 0.99), because at that threshold it selects 1.8× as many guards; the encoder's contribution appears once the two are placed at matched guard budget (Section 6.2), and directly at a common threshold only out of distribution (Table 9). All rows are the displayed seed (1234); Table 3 gives the four-seed mean±std.

misleading, because masking the embedding also changes how many vertices the probe selects: at $t = 0.20$ the ablation returns nearly twice the full probe's guard fraction (Table 5). Coverage-failure counts then track the guard budget rather than the representation, and across the $2 \times 2$ ablation the number of polygons below the 0.99 gate falls monotonically as the selected set grows (Table 5) — so read at fixed $t$ this ablation measures set size, not what the encoder represents. We report it in that form for completeness and do not draw the encoder's contribution from it.

The encoder's contribution is instead read at *matched guard budget*. We sweep all four conditions over $t \in [0.05, 0.80]$ — wide enough that their cost curves overlap despite spending very differently at any common $t$ — and compare them where they spend alike (Table 6). Over that band the ordering is strict and seed-stable: the tail grows monotonically as inputs are removed, full

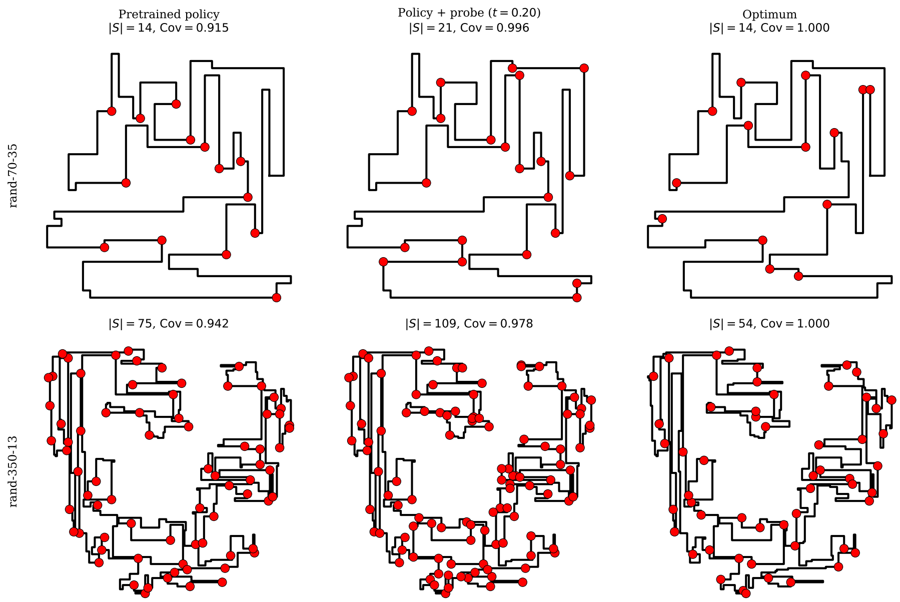


Figure 3: Two polygons drawn with the policy's seed (left column), the policy plus the SetPredictor probe at $t = 0.20$ (middle column), and the optimum (right column). Vertices selected as guards are marked. Both are cases the probe rescues, and both are chosen by a stated rule rather than by eye. Row 1 is in-distribution ($n = 70$): the seed falls below the 0.95 coverage gate ($\mathrm{Cov}_P(\cdot) = 0.915$) and the probe lifts it above (0.996) at $|S|/\mathrm{OPT}$ 1.50. It is the median-by-$n$ member of the 57 `test` polygons (of 362) where the seed fails the gate and the probe clears it — the median of that set, not its best case — and its cost sits below the split mean of 1.62 (Table 8). Row 2 is out-of-distribution ($n = 350$, about 1.8× the largest training polygon), where the seed again falls below the gate (0.942) and the probe lifts it above (0.978) at $|S|/\mathrm{OPT}$ 2.02, close to that split's mean of 1.95 (Table 9).

< no-seed < no-encoder < coords-only, in all sixteen probe-seed-by-budget cells. One point needs no interpolation at all: at $t = 0.30$ the full probe uses fewer guards than the no-encoder ablation *and* leaves about two-fifths fewer polygons below the 0.99 gate, dominating on both axes at once so that no trade-off has to be adjudicated. Read in the other direction the margin is near-constant across the band, which is why we quote it that way: to leave the same tail, the no-encoder ablation must spend about 1.45× the full probe's guards and coordinates alone about 1.75×, whereas masking only the seed costs about 1.1×. This is the measurement behind C1: it is the frozen encoder, not the policy's seed and not the coordinates, that lets the probe convert guards into coverage economically.

The comparison so far varies the probe seed on one policy, so we repeated it on the three independently trained policies of Section 6.4, sweeping each policy's own pair of probes over the same threshold range (Table 6c). The ordering survives the change of policy: at every matched budget both arms can reach, the encoder-bearing probe leaves the shorter tail — eleven of eleven reachable policy-by-budget cells, with the ablation leaving 1.5 to 5.4× as many polygons below the gate. Stated as cost, removing the encoder demands 1.30–1.81× the guards for the same tail across all four policies. C1 therefore rests on four training runs rather than one.

Two caveats keep this from being read more strongly than it should be. The claim is about *matched cost*, not about attainable coverage: given enough guards the no-encoder ablation does reach essentially complete coverage with no polygon below the gate, but it pays more than twice the full probe's guard cost to get there (Table 6a). And at a fixed threshold the comparison runs the other way: at $t = 0.20$ the ablation clears the 0.95 gate on every polygon while the full probe misses a handful (Table 4), because at that threshold it is spending nearly twice the guards to do so.

The no-encoder ablation removes the encoder but keeps the policy's seed indicator among the probe's inputs, so the recovered coverage could in principle be credited to the decoder's guard set rather than to the encoder. We therefore complete the $2\times2$ design, masking the seed indicator as well (Table 5 at $t = 0.20$; Table 6 for the swept curves). The cost multiples just quoted settle it: read at matched budget the two inputs are not comparable in importance. Masking the seed barely moves the probe, which traces almost the same coverage–cost curve; masking the encoder instead costs roughly 45% more guards for the same tail, and masking both roughly 75% more. The seed is not what carries the signal.

The coords-only condition is the sharpest of the four, because it also answers a natural baseline question: it is the same 464K-parameter Transformer, trained on the same LS targets with the same schedule, reading vertex coordinates alone with the encoder channels masked to zero. It never becomes economical. Matching the full probe's tail costs it about 1.75× the guards, and held to the full probe's budget it fails on more than three times as many polygons (Table 6b). Its apparently short tail at $t = 0.20$ is bought by selecting nearly three-quarters of all vertices (Table 5), which is over-guarding rather than guard selection. The seed, then, contributes little, whereas the encoder is what makes economical coverage reachable at all: it is the representation, not the decoder's seed and not the raw coordinates, that lets the probe place few guards and still cover.

Table 6 also makes the confound visible directly, rather than only asserting it. In panel (a) the arms' cost curves *cross*: at $t = 0.05$ the no-encoder ablation is by far the most expensive condition ($|S|/\mathrm{OPT}$ 5.17 against the full probe's 2.01) and has the shorter tail, whereas by $t = 0.40$ it has become the *cheaper* of the two (1.37 against 1.41) and has the longer tail (200 polygons below the gate against 96). Any single threshold therefore samples the four conditions at incomparable operating points, and which condition looks better is decided by which threshold is chosen — exactly the objection raised in review, and the reason panel (b) holds the budget fixed instead of the threshold.

Since the SetPredictor is itself a small Transformer, one might worry that *it* computes the geometry and the encoder merely passes coordinates through. To rule this out we replace the probe with a plain linear classifier (no hidden layers, no attention) that reads only the frozen per-vertex features and scores

| Probe inputs | #{Cov < 0.99} | min Cov | $\|S\|/n$ | $\|S\|/\text{OPT}$ |
|---|---|---|---|---|
| SetPredictor full | $57.8 \pm 4.8$ | $0.551 \pm 0.327$ | $0.25 \pm 0.01$ | $1.62 \pm 0.03$ |
| no-seed (encoder only) | $45.5 \pm 13.4$ | $0.547 \pm 0.335$ | $0.30 \pm 0.02$ | $1.94 \pm 0.12$ |
| no-encoder (seed only) | $22.0 \pm 6.0$ | $0.941 \pm 0.023$ | $0.45 \pm 0.04$ | $2.94 \pm 0.27$ |
| coords-only (neither) | $0.8 \pm 0.8$ | $0.962 \pm 0.044$ | $0.72 \pm 0.01$ | $4.64 \pm 0.06$ |

Table 5: Encoder–seed ablation on `test` (362 polygons), all probes at $t = 0.20$, four-seed mean ± std over $\{1234, 11, 22, 33\}$. The four rows are the $2 \times 2$ on/off combinations of the two probe inputs: the frozen encoder embedding and the policy's seed indicator. An absent input is masked to zeros, leaving architecture and parameter count unchanged. The two axes must be read together, the coverage tail (#{Cov < 0.99}, min Cov) against guard cost ($|S|/n$, $|S|/\text{OPT}$), because the same threshold $t = 0.20$ does not produce the same operating regime across rows: a probe that selects nearly all vertices will show a short tail simply by over-guarding, not by genuinely learning to place guards. The encoder-only probe stays close to the full probe on both. The seed-only probe shows a *shorter* tail here than the full probe, but only by spending 1.8× the guards ($|S|/n$ 0.45 against 0.25); at matched cost the ordering reverses (Table 6, and Section 6.2), which is why the encoder comparison is not drawn from this column. The coords-only probe holds coverage here only by *flooding*: at this threshold it selects $|S|/n = 0.72$, nearly three-quarters of all vertices, so its small #{Cov < 0.99} reflects set size rather than selection quality. At $t = 0.20$, then, only the encoder-bearing probes sit in the economical regime ($|S|/\text{OPT}$ 1.6–1.9) while the others sit at 2.9× (seed only) and 4.6× (neither); this is a difference in where the threshold places each probe, and the cost of the missing encoder is quantified at matched budget in Table 6.

guard against non-guard vertices. It already reaches ROC-AUC ≈ 0.84, a measure of how reliably the rule ranks a guard above a non-guard (0.5 is chance, 1.0 perfect).[2] A linear readout cannot be credited with computing geometry itself [9, 10], so the guard-relevant structure must already be linearly accessible in the frozen embedding: the encoder, not the probe, carries it. Scored by such a linear rule, guard vertices pile up at higher scores than non-guards, with only modest overlap, as visible in Figure 4.

*Probe capacity.* The SetPredictor is an expressive model, so we should separate what the representation contains from what the probe computes on top of it [10, 12]. A capacity ladder over the same frozen features and held-out protocol does this (Table 7): a linear readout, an attention-free MLP (the SetPredictor with its self-attention blocks removed, leaving a per-vertex projection with masked-mean-pooled seed and global context), a single self-attention layer, and the full three-layer probe. Per-vertex ROC-AUC rises with capacity but saturates early (Table 7). The attention-free MLP recovers about 84% of the linear-to-full gain; the two attention rungs add only the small remainder. A modest nonlinear readout of the frozen embedding is thus enough, and the full Transformer's capacity adds little. The guard-relevant structure sits in the encoder; the probe does not build it. We compare capacities on ROC-AUC, which is independent of the threshold, because a fixed threshold puts different-capacity

[2] $L_2$-regularized logistic regression under 5-fold polygon-grouped cross-validation over 200 polygons / 12,424 vertices: ROC-AUC = $0.843 \pm 0.009$ and PR-AUC = $0.582 \pm 0.027$ (mean ± std across folds) against a 0.16 positive-class prior.

| | full | | no-seed | | no-encoder | | coords-only | |
|---|---|---|---|---|---|---|---|---|
| $t$ | $\lvert S\rvert$/OPT | #fail | $\lvert S\rvert$/OPT | #fail | $\lvert S\rvert$/OPT | #fail | $\lvert S\rvert$/OPT | #fail |
| *(a) swept operating curves* | | | | | | | | |
| 0.05 | 2.01 | 28 | 2.63 | 20 | 5.17 | 0 | 5.45 | 0 |
| 0.10 | 1.81 | 40 | 2.29 | 28 | 4.40 | 2 | 5.04 | 0 |
| 0.20 | 1.62 | 58 | 1.94 | 46 | 2.94 | 22 | 4.64 | 1 |
| 0.30 | 1.50 | 74 | 1.74 | 64 | 1.77 | 126 | 4.31 | 4 |
| 0.40 | 1.41 | 96 | 1.58 | 87 | 1.37 | 200 | 3.68 | 18 |
| 0.50 | 1.32 | 119 | 1.45 | 114 | 1.21 | 239 | 2.43 | 94 |
| 0.60 | 1.23 | 153 | 1.33 | 150 | 1.13 | 255 | 1.25 | 278 |
| 0.80 | 0.98 | 269 | 1.08 | 254 | 1.06 | 284 | 0.24 | 343 |
| *(b) #fail at matched guard budget* $\lvert S\rvert$/OPT | | | | | | | | |
| 1.95 | $32 \pm 5$ | | $43 \pm 1$ | | $101 \pm 14$ | | $175 \pm 12$ | |
| 1.70 | $50 \pm 6$ | | $69 \pm 4$ | | $136 \pm 14$ | | $215 \pm 9$ | |
| 1.50 | $76 \pm 7$ | | $101 \pm 5$ | | $170 \pm 10$ | | $245 \pm 8$ | |
| 1.30 | $126 \pm 12$ | | $159 \pm 6$ | | $216 \pm 3$ | | $270 \pm 9$ | |
| *(c) guards the no-encoder arm needs for the same tail, per policy* | | | | | | | | |
| policy | #fail=30 | | #fail=50 | | #fail=76 | | #fail=100 | |
| 1234* | — | | 1.45× | | 1.45× | | 1.42× | |
| 11 | — | | — | | 1.33× | | 1.34× | |
| 22 | 1.38× | | 1.38× | | 1.31× | | 1.30× | |
| 33 | 1.81× | | 1.80× | | 1.78× | | 1.72× | |

Table 6: Encoder–seed ablation at *matched guard budget* on `test` (362 polygons) — the companion to Table 5, which fixes the threshold instead. Four-seed mean over $\{1234, 11, 22, 33\}$; #fail is the number of polygons below the 0.99 coverage gate, scored by exact CGAL. *(a)* each condition swept over $t \in [0.05, 0.80]$, wide enough that the four cost curves overlap and cross; the columns are not comparable row-wise, which is the point. *(b)* each condition read at four common guard budgets, interpolated along *its own* curve, so no condition is evaluated at another's threshold; entries are mean $\pm$ std over the four seeds. Panel (b) is where the encoder comparison is made: the ordering full < no-seed < no-encoder < coords-only is strict in all 16 seed-by-budget cells.

probes at different operating points (the calibration caveat of Table 6).

Closing the tail has a price, with the threshold $t$ as the knob: guard cost rises monotonically as $t$ falls, and among the three reported thresholds it is largest at the headline $t = 0.20$ ($|S|$/OPT 1.62, against 1.09 for the policy alone). Table 8 traces the knob over its whole range on the same held-out split, and the shape matters for how the method should be read. Guard cost falls steeply as $t$ rises while coverage decays slowly: by $t = 0.50$ the probe is at $|S|$/OPT 1.32 and still averages 0.984 coverage, clearing the 0.95 gate on 337 of 362 polygons against the policy's 293 — far cheaper than the headline operating point and still well ahead of the seed. The gain is bounded, though, and the curve says where it stops: by $t = 0.70$ the probe has converged to the policy seed on both axes ($|S|$/OPT 1.13 against 1.09, mean coverage 0.966 against 0.969), and by $t = 0.80$ it is strictly worse than the seed — cheaper at 0.98, but covering less on every measure, including 245 polygons past the gate against 293. The headline threshold is thus one end of a genuine trade-off rather than the only place the probe is usable, and the tail-closing behaviour that motivates it (Table 4) is

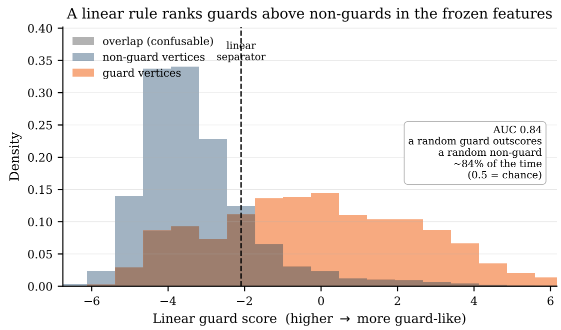


Figure 4: Guard-relevant structure is linearly accessible in the frozen encoder features. A one-dimensional linear rule (LDA) over the same 200-polygon / 12,424-vertex probe set scores each vertex; the histograms show that score for guard and non-guard vertices separately. The rule is fit and scored *out-of-fold* (GroupKFold over polygons — no vertex is scored by a rule that saw its own polygon), so the effect reflects embedding structure rather than memorization. Guards score higher, corresponding to ROC-AUC $\approx$ 0.84 and matching the logistic probe in the text. The classes are *not* linearly separable: the shaded band is the overlap the rule confuses, and it is substantial. What the frozen features support is a useful *ranking* of vertices by guard-relevance, which is the property the probe exploits and, as Section 7 argues, also the reason a per-vertex threshold must over-select to guarantee coverage.

| Probe capacity | Params | ROC-AUC | PR-AUC |
|---|---|---|---|
| Linear (logistic) | 129 | $0.837 \pm 0.000$ | $0.531 \pm 0.000$ |
| Attention-free MLP | 66,561 | $0.909 \pm 0.001$ | $0.633 \pm 0.003$ |
| 1 self-attention layer | 199,041 | $0.912 \pm 0.003$ | $0.643 \pm 0.011$ |
| Full SetPredictor (3 layers) | 464,001 | $0.923 \pm 0.002$ | $0.658 \pm 0.015$ |

Table 7: Probe-capacity ladder: held-out per-vertex ROC-AUC / PR-AUC on the 362-polygon `test` split against the LS target. Attention-bearing rungs are four-seed mean $\pm$ std; the linear rung is a single held-out logistic fit (consistent with the cross-validated 0.843 in the text). Most of the linear-to-full gain arrives with no self-attention at all.

bought at the curve's expensive end. Figure 5 gives the full picture in and out of distribution: the coverage CDF sits to the right of the policy at every threshold, while the $|S|/n$ and $|S|/\mathrm{OPT}$ box plots show the matching cost across all three thresholds. We report the trade-off rather than fix one operating point.

### *6.3. Out-of-distribution generalization*

The next test of the representation is the `ood` split, a larger held-out set that spans the training size range and extends well beyond it, reaching $n = 1000$, about five times the largest training polygon. The policy alone drops in mean coverage, clearing the 0.95 gate on 85% of the 2081 polygons; the probe at $t = 0.20$ lifts that to 99%, roughly an order-of-magnitude reduction in failures (Table 9). The improvement is significant for every probe seed individually (exact McNemar test on the paired per-polygon outcomes, $p < 10^{-74}$ in all four cases), and the per-seed Wilson 95% intervals for the gate-clearing rate all lie above 0.977. Nor is it an artifact of the in-distribution-size majority of the split:

| $t$ | Mean $\mathrm{Cov}_P(S)$ | Mean $\lvert S\rvert/n$ | Mean $\lvert S\rvert/\mathrm{OPT}$ | $\#\{\mathrm{Cov}_P(S) \geq 0.95\}$ |
|---|---|---|---|---|
| *policy seed* | 0.9689 | 0.166 | 1.09 | 293 |
| 0.05 | $0.9959 \pm 0.0016$ | $0.304 \pm 0.005$ | $2.01 \pm 0.03$ | $357.2 \pm 1.5$ |
| 0.10 | $0.9942 \pm 0.0020$ | $0.275 \pm 0.005$ | $1.81 \pm 0.03$ | $353.2 \pm 2.8$ |
| 0.15 | $0.9925 \pm 0.0027$ | $0.259 \pm 0.004$ | $1.70 \pm 0.03$ | $351.5 \pm 3.2$ |
| 0.20* | $0.9912 \pm 0.0024$ | $0.247 \pm 0.005$ | $1.62 \pm 0.03$ | $349.8 \pm 3.5$ |
| 0.25 | $0.9903 \pm 0.0026$ | $0.238 \pm 0.005$ | $1.56 \pm 0.03$ | $348.0 \pm 3.3$ |
| 0.30 | $0.9893 \pm 0.0027$ | $0.230 \pm 0.006$ | $1.50 \pm 0.04$ | $346.5 \pm 3.2$ |
| 0.35 | $0.9883 \pm 0.0028$ | $0.223 \pm 0.006$ | $1.45 \pm 0.04$ | $345.2 \pm 4.0$ |
| 0.40 | $0.9869 \pm 0.0027$ | $0.216 \pm 0.007$ | $1.41 \pm 0.04$ | $343.2 \pm 3.5$ |
| 0.45 | $0.9852 \pm 0.0026$ | $0.209 \pm 0.007$ | $1.36 \pm 0.04$ | $339.2 \pm 4.6$ |
| 0.50 | $0.9836 \pm 0.0029$ | $0.203 \pm 0.007$ | $1.32 \pm 0.04$ | $336.8 \pm 5.1$ |
| 0.55 | $0.9809 \pm 0.0042$ | $0.197 \pm 0.007$ | $1.28 \pm 0.04$ | $331.8 \pm 7.9$ |
| 0.60 | $0.9773 \pm 0.0051$ | $0.190 \pm 0.007$ | $1.23 \pm 0.05$ | $326.5 \pm 9.4$ |
| 0.70 | $0.9664 \pm 0.0067$ | $0.174 \pm 0.008$ | $1.13 \pm 0.05$ | $304.2 \pm 15.4$ |
| 0.80 | $0.9376 \pm 0.0145$ | $0.151 \pm 0.010$ | $0.98 \pm 0.06$ | $244.5 \pm 35.4$ |

Table 8: The probe's full coverage–cost operating curve, single pass, on the held-out `test` split (362 polygons), four-seed mean $\pm$ std, with the policy seed as reference. Same population and protocol as Tables 3 and 6, so rows are directly comparable across them; * marks the headline threshold. The policy row carries no spread because it is the same released policy in all four columns. The shape is the point: guard cost falls steeply as $t$ rises while coverage decays slowly, until the probe converges to the policy seed on both axes near $t = 0.70$ and is strictly worse than it by $t = 0.80$.

| Method | Mean cov | $\#\{\mathrm{Cov} \geq 0.95\}/N$ | Mean $\lvert S\rvert/n$ | Mean $\lvert S\rvert/\mathrm{OPT}$ |
|---|---|---|---|---|
| Pretrained pointer (seed) | 0.9567 | 1778/2081 (0.839,0.869) | 0.1807 | 1.1488 |
| SetPredictor full ($t = 0.20$) | $\mathbf{0.9936} \pm 0.0009$ | $2058.2 \pm 7.3/2081$ | $0.3023 \pm 0.0055$ | $1.9486 \pm 0.0363$ |
| SetPredictor, no-encoder ($t = 0.20$) | $0.9874 \pm 0.0023$ | $2047.8 \pm 4.7/2081$ | $0.3670 \pm 0.0335$ | $2.3571 \pm 0.2149$ |

Table 9: Out-of-distribution evaluation on `ood` (2081 polygons, $n$ up to 1000). Rows as in Table 3: the policy seed, the full probe, and the no-encoder ablation, both probes the four-seed mean $\pm$ std at $t = 0.20$.

on the genuinely larger-than-training polygons alone ($n > 198$, 885 instances) the policy clears the gate on 85% and the probe on 98.7%. This is the evidence behind C2, and Section 6.4 shows it is not specific to the policy we released. What is *not* stable is the residual failure count, which varies by a factor of nearly three across probe seeds (Table 9).

The encoder comparison needs more care here. On this policy the encoder gains on *both* axes at a common threshold, which it does not do in distribution: at $t = 0.20$ the full probe uses fewer guards than the no-encoder ablation *and* leaves fewer polygons below the gate (Table 9). That double gain is specific to this policy, however. Repeating the same fixed-threshold comparison on three independently trained policies reproduces it on one and reverses the tail on the other two, where the ablation again clears more polygons by spending more guards (Section 6.4). The guard-budget confound therefore does not reliably lift out of distribution; it lifted on the policy we happened to release. What generalizes is the matched-budget reading, which holds on all four policies in distribution (Table 6c). Out of distribution we swept the full threshold range

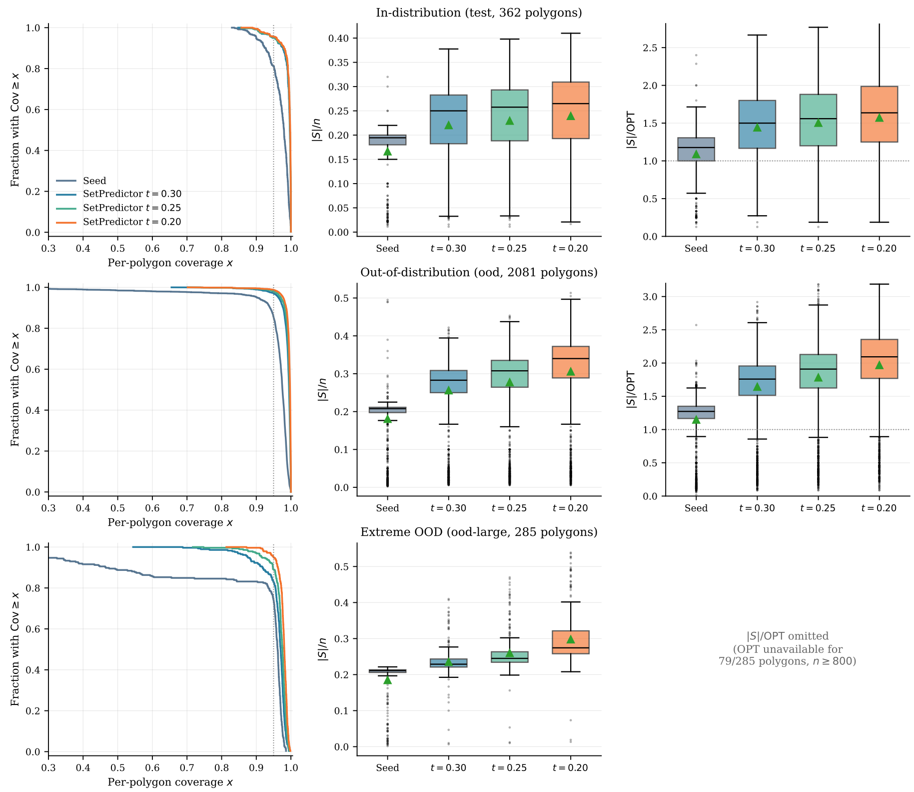


Figure 5: Distributional view across all three regimes: `test` (top, 362 polygons), `ood` (middle, 2081), and `ood-large` (bottom, 285, $n$ up to 2250). *Left:* per-polygon coverage as a complementary stair-step CDF; the dashed line marks the 0.95 coverage gate. *Center:* box plots of $|S|/n$ for the policy seed and the three probe thresholds. *Right:* the approximation ratio $|S|/\mathrm{OPT}$ for `test` and `ood` only; omitted for `ood-large`, where OPT is unavailable for the larger polygons (Section 5.3). Boxes show the displayed seed (1234); four-seed mean±std is in Tables 3 and 9.

for the released policy only, and there the matched-budget margin is wider than the fixed-threshold one. Figure 5 (middle and bottom rows) shows the matching coverage and cost distributions.

| Method | Mean Cov | min Cov | $\#\{\mathrm{Cov} \geq 0.95\}/N$ | Mean $\lvert S\rvert/n$ |
|---|---|---|---|---|
| Pretrained pointer (seed) | 0.8678 | 0.038 | 210/285 | 0.1849 |
| SetPredictor full ($t = 0.20$) | $\mathbf{0.9734} \pm 0.0064$ | $0.766 \pm 0.106$ | $265.8 \pm 10.4/285$ | $0.2890 \pm 0.0178$ |
| SetPredictor, no-encoder ($t = 0.20$) | $0.9379 \pm 0.0085$ | $0.105 \pm 0.079$ | $255.8 \pm 5.5/285$ | $0.3069 \pm 0.0223$ |

Table 10: Extreme out-of-distribution evaluation on `ood-large` (285 polygons, $n$ from 600 to 2250), coverage scored by exact CGAL at $t = 0.20$ over all 285. The approximation ratio is omitted because OPT is unavailable for 79 polygons ($n \geq 800$; Section 5.3). The full-probe and no-encoder rows are four-seed mean $\pm$ std; their min Cov is the mean $\pm$ std of the four per-seed worst-polygon coverages.

*Extreme out-of-distribution.* On the `ood-large` split ($n$ from 600 to 2250, up to about eleven times the largest training polygon), the worst-case coverage is the more informative metric, as the mean can look reasonable while the tail collapses (Table 10). The pretrained seed leaves its worst polygon almost entirely uncovered, and the no-encoder ablation barely moves it: its worst-polygon coverage stays collapsed on every probe seed. The full probe lifts that worst polygon on every seed, to roughly seven times the ablation's worst-case coverage on the four-seed mean, though still well short of the gate (Table 10). This is where the encoder's contribution is clearest at this scale, and it is the only axis on which it is clear. Neither arm clears the gate everywhere: the full probe is ahead on three of the four probe seeds, but the margin sits inside its own seed spread, so we do not read a gate-count advantage from this split. Nor does the probe buy a guard saving here, unlike on the other two splits — the two arms' approximation ratios differ by less than the seed spread. Eleven times beyond the training range, then, the encoder's advantage is confined to the worst case.

### *6.4. Replication across independently trained policies*

Everything above reads one policy. Because the paper's question is what *a* reinforcement-trained encoder holds, a single training run cannot separate the claim from the run, so we trained three further PO/BT policies from scratch under the identical recipe (seeds 11, 22, 33; Section 5.2) and, for each, trained its own full and no-encoder probes on its own local-search targets. A probe is only meaningful relative to the encoder it reads, so the policies cannot share probes. Table 11 reports all four policies on all three splits.

Two things replicate and one does not. The four policies differ appreciably in quality, both in gate-clearing and in approximation ratio (Table 11), so this is a genuine sample of the recipe rather than a re-run of one outcome. First, the probe closes the coverage tail on *every* policy, and by the largest margins out of distribution, where each policy's below-gate count falls by roughly an order of magnitude or more. This is the evidence behind C2, and it now rests on four training runs. Second, the matched-budget encoder comparison of Section 6.2 survives the change of policy (Table 6c). What does *not* replicate is the fixed-threshold encoder comparison out of distribution, discussed above.

One row needs reading with care. On `ood-large` policy 22 does not decode a guard set so much as flood one: its seed already selects nearly three-quarters of every polygon's vertices and trivially covers all 285 (Table 11). It has no tail to close, and the probe — which selects *fewer* guards — necessarily gives coverage back. We exclude that cell from the encoder comparison rather than scoring it as a failure of the probe, on the same reasoning that makes a fixed threshold the wrong place to compare arms: a method that already spends most of the budget cannot be beaten on coverage, and beating it there would mean nothing. Among the three policies that do leave a tail on this split, the full probe leaves the shorter one in all three.

| Policy | policy seed $|S|/n$ | policy seed gate | + full probe $|S|/n$ | + full probe gate | + no-encoder $|S|/n$ | + no-encoder gate |
|---|---|---|---|---|---|---|
| `test` (362 polygons) | | | | | | |
| 1234* | 0.166 | 293/362 | 0.239 | 345/362 | 0.424 | 362/362 |
| 11 | 0.160 | 290/362 | 0.222 | 342/362 | 0.419 | 362/362 |
| 22 | 0.171 | 286/362 | 0.260 | 353/362 | 0.380 | 361/362 |
| 33 | 0.169 | 264/362 | 0.308 | 359/362 | 0.498 | 361/362 |
| `ood` (2081 polygons) | | | | | | |
| 1234* | 0.181 | 1778/2081 | 0.306 | 2055/2081 | 0.329 | 2041/2081 |
| 11 | 0.173 | 1688/2081 | 0.299 | 2051/2081 | 0.371 | 2075/2081 |
| 22 | 0.298 | 1861/2081 | 0.372 | 2053/2081 | 0.424 | 2074/2081 |
| 33 | 0.222 | 1682/2081 | 0.377 | 2078/2081 | 0.402 | 2050/2081 |
| `ood-large` (285 polygons) | | | | | | |
| 1234* | 0.185 | 210/285 | 0.298 | 270/285 | 0.277 | 247/285 |
| 11 | 0.200 | 200/285 | 0.283 | 278/285 | 0.316 | 271/285 |
| 22 | 0.719 | 285/285 | 0.586 | 222/285 | 0.697 | 262/285 |
| 33 | 0.569 | 220/285 | 0.585 | 266/285 | 0.529 | 119/285 |

Table 11: Replication across four independently trained PO/BT policies, $t = 0.20$, coverage by exact CGAL. Each policy has its own full and no-encoder probe trained on its own targets; * marks the released policy used everywhere else in the paper. This table varies the *policy* seed, whereas Tables 3, 9 and 10 vary the *probe* seed at policy 1234 — the two are different quantities and are not pooled. "gate" is $\#\{\mathrm{Cov}_P(S) \geq 0.95\}/N$. Guard cost is given as $|S|/n$ throughout because OPT is unavailable for 79 of the `ood-large` polygons. The two columns must be read together: at this fixed threshold the no-encoder arm often clears more polygons while spending far more guards, which is the confound Table 6 resolves, and the reason the encoder conclusion is drawn at matched budget rather than from this table. On `ood-large`, policy 22's seed already selects $|S|/n = 0.719$ and covers everything, so that policy has no tail to close on that split (see text).

### *6.5. Alternatives to the probe: decode search, editors, and the fixed point*

*Decode-time search does not close the tail.* The simplest alternative to the probe is to decode the policy better rather than read its encoder. We test this directly (Table 12): for each `test` polygon we draw $K = 32$ stochastic rollouts from the frozen policy and select among them geo-free, by length-normalized log-likelihood, with no visibility oracle. The result is statistically indistinguishable from greedy (Table 12) because the policy's maximum-likelihood decode already *is* the greedy seed: likelihood-ranked search never prefers a more-covering set, and occasionally prefers a less-covering one, lowering the worst polygon. Letting an *oracle* pick the highest-coverage sample of the 32 (classical active search, no longer geo-free) raises gate-clearing but still leaves a substantial tail below 0.99 (Table 12), far short of what reading the encoder achieves (Table 4). The tail is thus not a decoding artifact that more search removes: closing it requires reading the encoder, and the probe pays for that in guards rather than in search.

Before settling on the single-shot probe we tried to improve the seed with a learned *iterative* editor applying ADD/REMOVE/STOP edits. Three architectures of increasing capacity, including one with topology features and an auxiliary visibility-prediction loss, and one with DAgger refresh [32], failed to improve over the seed in any coverage-preserving regime. The failure takes the form

| Decode of the frozen policy | $\#\{\mathrm{Cov} \geq 0.95\}/N$ | $\#\{\mathrm{Cov} < 0.99\}$ | Mean Cov | min Cov | $\lvert S\rvert/n$ | $\lvert S\rvert/\mathrm{OPT}$ |
|---|---|---|---|---|---|---|
| greedy (seed) | 293/362 | 265 | 0.969 | 0.830 | 0.166 | 1.09 |
| best-of-32, likelihood (geo-free) | 294/362 | 266 | 0.967 | 0.557 | 0.166 | 1.09 |
| best-of-32, coverage oracle (*not* geo-free) | 333/362 | 199 | 0.981 | 0.845 | 0.171 | 1.12 |

Table 12: Decode-time search on `test` (362 polygons), all reading the same frozen PO/BT policy. *greedy* is the seed of Table 3. *best-of-*32*, likelihood* draws $K = 32$ stochastic rollouts and keeps the one with the highest length-normalized log-likelihood — a geo-free decode search with no visibility oracle. *best-of-*32*, coverage oracle* instead keeps the highest-coverage sample (tie-break: fewer guards); it consults exact coverage to choose and is therefore *not* geo-free, reported only as an upper bound on what the policy's own samples contain. Greedy is included in each candidate pool, so neither selection can underperform it on its own ranking metric. Geo-free search matches greedy and does not close the tail; even oracle-assisted search falls well short of the SetPredictor (Table 4).

of a trade the editor cannot escape. Its best configuration preserves coverage ($0.969 \to 0.972$ mean) but cuts only 13% of the guards, and matches or beats the local-search reference on both axes for just 0.27 of the polygons it edits — a *recovery rate* well below replacement quality. Configurations tuned to cut harder do cut harder, up to 26%, but pay for it in coverage, dropping the mean to 0.925. Neither end of that trade is usable, which is what we mean by no coverage-preserving regime. Three structural reasons recur: errors accumulate because the editor sees clean trajectories in training but its own predictions at inference; a single $\{\text{ADD}, \text{REMOVE}, \text{STOP}\} \times \{1, \ldots, n\}$ head couples selection with termination, miscalibrating STOP; and the train/inference state distributions diverge in ways DAgger refresh did not close.

The same teacher-forcing cascade also defeats a *supervised* pointer network, which we trained directly on AGPVG solutions before adopting the reinforcement recipe. Cloning a guard sequence is ill-defined for a second reason beyond the cascade: a polygon has many optimal and near-optimal guard sets (the constraint $\mathrm{Cov}_P(S) = 1$ in Eq. (2) fixes the cardinality, not the membership), so any single target sequence is an arbitrary choice among equivalent solutions, and an autoregressive loss penalizes the model for proposing a different but equally valid one. Forced onto sequences it would not itself generate, the recurrent decoder then drifts into hidden states unseen during training and collapses (mean coverage $\approx 0.50$ at roughly $3\times$ the optimum), the brittleness of supervised cloning anticipated in Section 1. Preference optimization sidesteps both problems: it ranks the policy's *own* rollouts by reward rather than matching a designated target, so non-unique optima are not in conflict. The single-shot probe likewise has none of these failure modes: no rollout, no stop token, and a per-vertex membership label rather than an ordered sequence, trained and used in the same regime.

A clarification this invites: the probe's supervised targets $S^\star_{\mathrm{LS}}$ are themselves the output of this same edit family run to its best-improvement optimum: an *oracle* editor seeded by the policy. So the contrast is narrower than "editors fail, the probe works". The probe does not reproduce the oracle's guard sets: at the headline threshold it selects roughly $1.8\times$ the cardinality of the very sets it was trained to predict (Table 3), and raising its threshold to match that cardinality costs coverage rather than recovering the oracle's solution (Table 8). What it reproduces is the oracle's *ranking* of vertices, not its judgement of how many to keep; it reaches comparable coverage by over-selecting. That the probe succeeds at all is therefore not, by itself, evidence about the encoder, since it could merely be distilling the oracle's preferences. What attributes the success to the *representation* is the encoder–seed ablation of Section 6.2 (Table 5), run with the LS target held fixed: masking the seed indicator barely moves the probe, masking the frozen embedding markedly widens the tail, and removing both leaves a probe that stays above the gate only by flooding guards; a zero-capacity linear probe over the same embedding already ranks guards above non-guards. Were the probe merely distilling the oracle, which inputs it reads would not matter, yet it is the encoder, not the decoder's seed, that makes the targets predictable from coordinates alone.

Re-running the probe with its own output as the updated seed indicator leaves the prediction nearly unchanged over most of the threshold range, and the residual drift has a consistent direction rather than being noise: at every threshold tested, five passes end with fewer guards and slightly lower coverage than one (Figure 6). The pass-by-pass movement is monotone wherever that drift is appreciable ($t \geq 0.6$); at $t = 0.5$ it is not, but there the total movement is within run-to-run noise ($|S|/\mathrm{OPT}$ varies by 0.003 across $K$). The size of the drift depends strongly on the threshold. At $t = 0.5$ the probe is a fixed point for practical purposes ($|S|/\mathrm{OPT}$ moves 0.0045 between $K = 1$ and $K = 2$, coverage 0.0011 over $K = 1 \to 5$); by $t = 0.65$ the $K = 1 \to 2$ step moves $|S|/\mathrm{OPT}$ by 0.029, and at $t = 0.8$ — where the selected set is smallest and the test most stringent — it moves 0.079 with coverage falling 0.041 over $K = 1 \to 5$. So the fixed-point property holds where the probe is operated, and degrades gracefully into a slow contraction as the threshold is pushed toward the decision boundary; it is not exact. We test the fixed point at this higher range $t \in \{0.5, \ldots, 0.8\}$ rather than at the headline $t = 0.20$ by design: a higher threshold selects fewer vertices, so the probe's output set is small and differs most from the large seed set it started from, which makes re-inserting it as the next pass's seed the *most* likely to move the prediction: the stringent test, nearest the decision boundary. At the headline $t = 0.20$ the selected set is large and changes even less under iteration, so the single-pass conclusion is only more firmly supported there. This is the fixed-point property (C4): with the seed indicator already among its inputs, a single pass suffices.

### *6.6. Runtime and memory: geo-free versus classical*

Geo-free inference moves the visibility cost from inference to training; Table 13 shows the effect on real polygons across the size grid used for the learned-pipeline timing. At inference the learned pipeline is one coordinate-only forward pass, a policy encode plus a probe pass, costing at most a few hundred milliseconds even at $n = 2000$ (same workstation as Section 5.3; on the GPU, the probe alone accounts for $\approx 10$ ms of that at $n = 2000$, versus $\approx 180$ ms on the CPU, where its $O(n^2)$ self-attention dominates rather than the sequential LSTM encode).

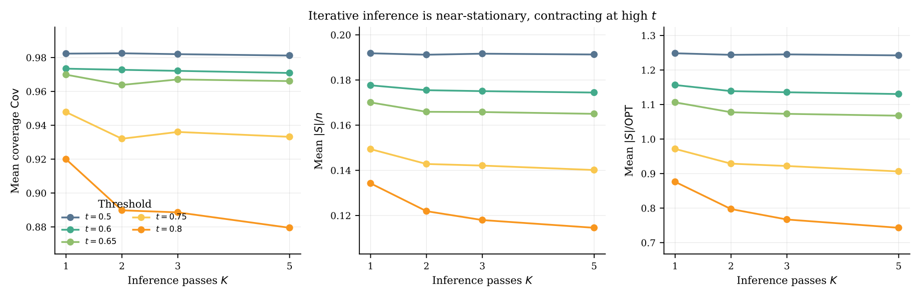


Figure 6: Iterative inference is near-stationary at the thresholds the probe is operated at, contracting slowly at the highest ones, on the held-out `test` split (362 polygons). Each panel shows mean coverage (*left*), $|S|/n$ (*centre*), and $|S|/\mathrm{OPT}$ (*right*) versus inference passes $K \in \{1, 2, 3, 5\}$ for thresholds $t \in \{0.5, 0.6, 0.65, 0.75, 0.8\}$ — a higher range than the headline thresholds, where the property is most stringently tested near the decision boundary. Drift has a consistent direction — $K = 5$ sheds guards and loses a little coverage relative to $K = 1$ at every threshold — and grows with $t$: within noise at $t = 0.5$, 0.079 in $|S|/\mathrm{OPT}$ at $t = 0.8$ (bounds in Section 6.5).

Table 13 reports two classical implementations, because the choice that matters for training also matters here. *Exact greedy* is the implementation used for every guard-count and coverage comparison elsewhere in the paper (Table 3 onward): each candidate's marginal gain is an exact CGAL polygon-set union, with the selection loop accelerated by lazy evaluation of the submodular marginal gains, which leaves the returned guard sets unchanged. *Disc-vis greedy* scores the same additive algorithm on the discretized visibility matrix of Eq. (4) instead, the same $M = 500$-sample approximation Section 4.1 uses for PO/BT rewards and the local-search targets, so ranking all remaining candidates is one vectorized array operation per round rather than a CGAL call per candidate. Neither escapes the structure both must build first: before a single guard is chosen, visibility precompute alone costs 20–28× the learned GPU pass for exact greedy and 56–74× for disc-vis.

The two classical variants differ in where their cost sits, not in whether they pay it. Exact greedy's selection step scales as $\approx n^{1.8}$ across the grid, in line with the $O(n^2)$ polygon-operation count it performs, and reaches $\approx 84$ s at $n = 2000$; disc-vis greedy's selection is close to free, leaving visibility construction to dominate its total. Measured end to end, exact greedy costs 220–1495× the learned GPU pass, and disc-vis greedy, the fastest classical alternative we could construct, still costs 16–28× the learned CPU pass and 57–75× the learned GPU pass. The geo-free advantage therefore holds against an optimized classical baseline, not merely a naive one.

That speed is not free. Disc-vis greedy stops the moment its own sample-based estimate crosses the same 0.99 gate used elsewhere, but scoring the resulting guard set with exact CGAL afterwards shows the two diverging as $n$ grows: by $n = 2000$ the estimate still reads near 0.99 while true coverage has fallen well below it (Table 14). The gap tracks guard count more than it tracks $n$, which points to a selection effect rather than plain sampling noise: each greedy round hill-climbs against the same fixed 500 points, so the more rounds it runs, the more chances it has to land on a guard set that scores well on those particular points without truly covering the polygon.[3] Training does not hit this the same way (the PO/BT reward is one noisy sample per rollout, averaged over many rollouts and polygons, and the local-search editor refines an already-good seed with a handful of edits rather than building a guard set from empty over dozens of rounds), which is why the paper relies on this approximation for training and local-search targets but always re-scores with exact CGAL at evaluation, rather than treating one sample-based estimate as ground truth for a single high-stakes decision.

On memory, the disc-vis matrix is $n \times 500$ booleans, 100 KB at $n = 200$ growing to 1 MB at $n = 2000$, while the learned pipeline is a fixed $\approx$0.83M parameters ($\approx$3.3 MB at single precision) plus $O(nd)$ activations, independent of $n$; peak process memory was comparable across $n$ for both classical implementations, dominated by fixed library overhead rather than by the growing exact polygon-set representation. None of this makes the learned method a better solver, since classical greedy wins on guard count regardless of which scoring rule it uses (Section 6.1), but it is the basis for the settings in Section 1: when many instances must be solved or a geometry backend is costly, paying the visibility cost once at training beats paying it per instance, whether the classical alternative is exact or approximate.

[3] Over a family-diverse sample of 40 polygons spanning $n \in [20, 1750]$, the Spearman correlation of the gap with guard count is 0.95 against 0.88 with $n$, and controlling for the other separates them (0.78 partial correlation for guard count, 0.29 for $n$). Two polygons at the same $n = 500$ make the point directly: one needing 3 guards has a gap of $+0.004$, one needing 56 has a gap of $+0.044$.

| | Exact greedy | | Disc-vis greedy | | Geo-free learned | | |
|---|---|---|---|---|---|---|---|
| $n$ | precompute (ms) | selection (s) | precompute (ms) | selection (ms) | precompute | CPU (ms) | GPU (ms) |
| 200 | 119 | 1.2 | 338 | 1.5 | — | 12 | 6 |
| 500 | 286 | 6.3 | 932 | 5.9 | — | 33 | 13 |
| 1000 | 620 | 20.4 | 1664 | 19.8 | — | 89 | 25 |
| 2000 | 1609 | 83.6 | 4196 | 77.9 | — | 264 | 57 |

Table 13: Per-instance inference cost, split by mechanism rather than by method: *precompute* builds a structure once; *selection* then produces the guard set, querying that structure once per candidate over $\approx n/6$ rounds for the two classical methods. That query is expensive for exact greedy (an exact CGAL polygon-set union per candidate, with the loop accelerated by lazy evaluation of the submodular marginal gains, which returns identical guard sets) and cheap for disc-vis greedy (one vectorized lookup against the precomputed $M = 500$-sample matrix). The geo-free pipeline has nothing to precompute, since it never reads a visibility structure at inference (Section 2.3); its CPU and GPU columns are each a single coordinate-only forward pass standing in for selection. Exact greedy's selection is given in seconds, unlike the other columns, since it runs orders of magnitude above them; its growth is discussed in the text.

| $n$ | guards | disc-vis coverage (est.) | exact coverage (true) | gap |
|---|---|---|---|---|
| 200 | $20.5 \pm 7.3$ | 0.991 | $0.973 \pm 0.007$ | $0.018 \pm 0.006$ |
| 500 | $38.2 \pm 18.6$ | 0.991 | $0.950 \pm 0.017$ | $0.041 \pm 0.015$ |
| 1000 | $75.3 \pm 9.0$ | 0.990 | $0.907 \pm 0.010$ | $0.083 \pm 0.010$ |
| 2000 | $139.0 \pm 10.4$ | 0.990 | $0.807 \pm 0.007$ | $0.183 \pm 0.007$ |

Table 14: Disc-vis greedy's stopping quality, mean $\pm$ std over 8/5/3/3 polygons per bucket. "disc-vis coverage" is the sample estimate it stops on (target 0.99); "exact coverage" is the same guard set scored by CGAL afterwards.

## 7. Discussion

The central measurement in this paper is what happens when we read the encoder rather than the decoder. Reading only the frozen embeddings, the coordinates, and the policy's seed indicator, with no visibility matrix or CGAL output, the SetPredictor shortens the in-distribution coverage tail by at least four-fold on every probe seed (Table 3) and cuts the out-of-distribution one by about an order of magnitude (Section 6.3), on all four policies we trained (Section 6.4). Two readings fit this. Either the encoder learned enough geometry that a small classifier over its output recovers coverage, or the decoder's EOS calibration was the bottleneck and the policy had already found the right vertices but stopped early. Both give the same headline: this policy internalized a representation holding more of the geometry than its decoder expressed. The encoder–seed ablation (Table 5) favours the encoder reading, since masking the seed barely changes the probe while masking the encoder widens the tail or forces over-guarding. Three measurements support this (Section 6.2). The no-encoder ablation, masking the embedding with everything else fixed, leaves a probe that cannot match the full probe's coverage at equal cardinality: at matched guard budget it must spend appreciably more guards to leave the same tail, and coordinates alone more still (Table 6). The no-seed control shows the encoder alone nearly recovers the full probe, so the seed does not carry the signal. And the linear probe (ROC-AUC $0.843 \pm 0.009$), with the capacity ladder of Table 7 behind it, shows the structure is already accessible from the frozen embedding through a modest readout, not built by the probe. Two caveats bound the claim. Successful probing shows the LS target is decodable from the features, which is necessary for the encoder to carry the geometry but does not prove it carries all of it, nor that the residual failures are solely decoder calibration; we give the encoder reading as the better-supported one, not the only one. And the word *geometry* needs qualifying, because geometry is invariant to rigid motions and this representation is not: rotating or reflecting a polygon costs a large fraction of the gate-clearing count (Section 8). What the probe reads is therefore guard-relevant structure expressed in the canonical presentation of the training ecosystem — enough to recover the coverage decision on instances written the same way, but not the orientation-independent notion of visibility a geometer would mean. We take this as a statement about what *this* training setup produces rather than a limit on what encoder probing can reveal, and it is why we regard equivariance as the most promising next step.

Recovering coverage has a price, and we report it rather than disguise it. The probe's guard sets grow towards twice the optimum at the expensive end of the threshold range and fall to near the policy's own cardinality at the cheap end (Table 8), and the $|S|/\mathrm{OPT}$ box plots of Figure 5 show that the whole distribution widens as the threshold falls, not just its mean. A deployment would pick one operating point on this curve; we leave it as a knob because the right point depends on how costly an extra guard is relative to an uncovered region.

That price is not incidental, and reading it together with the capacity ladder suggests where the limit actually lies. Coverage is a submodular function of the guard set (Section 2.2): the value of a vertex depends on what the already-chosen guards leave uncovered. An autoregressive pointer can represent that dependence, conditioning each pick on the partial solution, which is why the policy attains near-classical cardinality; the probe cannot, because it emits one probability per vertex and thresholds them independently. It can therefore rank vertices by guard-relevance but cannot reason about marginal gain, and the only way an independent rule can guarantee coverage is to select past the point where the marginal vertex is needed. The over-guarding is the signature of that constraint rather than a tuning artifact. Two observations line up with this reading: the probe selects roughly $1.8\times$ the cardinality of the local-search sets it is trained on while reproducing their ranking (Table 3), and the capacity ladder shows self-attention adding almost nothing over an attention-free readout (Table 7) — if the bottleneck were representational, attention over vertices is exactly the capacity that should have helped. We therefore read the evidence as showing that the frozen encoder carries guard-relevant structure that a modest readout recovers, while converting that structure into a *minimal* covering set needs the sequential conditioning the single-shot probe gives up. This is a statement about what a per-vertex readout can express, not a claim that the encoder lacks the remaining information, and it is the form of the limit our experiments can actually speak to.

Note that we are not claiming a better AGP solver. Classical greedy reaches near-full coverage at close to the optimal guard count on `test` (Table 3); the probe at $t = 0.20$ reaches comparable coverage but at about $1.6\times$ the optimum across four seeds. A greedy-then-local-search pipeline, the standard high-quality classical recipe, would be stronger still on cardinality; the *LS on policy seed* row of Table 3, which refines a guard set with the same add/remove/swap editor to clear the 0.95 gate on all 362 at lower cardinality still, already bounds what such refinement achieves on this data. The classical anchors win on cardinality at preserved coverage, and the policy and probe are not built to contest that. What we document is what is learnable by reinforcement when the model is denied geometric input at the moment of placing guards, and what part of that learning lives in the representation rather than the decoder. The contrast between the failed iterative editors (Section 6.5) and the single-shot probe is the cleanest expression of the point: removing the stop time and the rollout is exactly what lets the encoder's information surface.

## 8. Limitations

We acknowledge several limitations of the work, enumerated here for clarity; the first two are conceptual and the rest experimental.

1. **Scope of the "no geometric knowledge" claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time. *Training* does see visibility: the PO/BT reward is computed from discrete visibility coverage, and the probe's supervised targets are generated by local search using discrete visibility as a coverage gate. "No explicit geometric knowledge" must be read as "no geometric oracle at the moment of placing guards."
2. **The probe is supervised, not reinforcement.** The SetPredictor is trained by BCE against local-search-derived labels. It is not itself a reinforcement learner. We position it as a representation probe of the RL-trained encoder, not as an RL contribution.
3. **Held-out partition size.** `test` has 362 polygons. Wilson 95% CIs on the gate-clearing proportion are reported, but the absolute count is modest. The OOD evaluation on `ood` (2081 polygons) provides a much larger second held-out evidence base.
4. **Single instance ecosystem; OOD is mainly size extrapolation.** Although the evaluation spans several polygon families (random orthogonal, random-simple, von Koch, boundary-simple) and a wide size range, nearly all instances originate from one ecosystem (the AGPVG library [18] plus our same-format `random` supplement), so the generator and coordinate format are largely shared between training and evaluation. The out-of-distribution axis we stress is therefore predominantly *size* extrapolation (to $\approx 5\times$ the training maximum on `ood`, $\approx 11\times$ on `ood-large`), not a shift to a genuinely different polygon distribution or instance source. Whether the encoder's geometry transfers across instance ecosystems (procedurally different generators, or real floorplans) is untested and left to future work.

5. **Policy selection and seeds.** Except for the replication of Section 6.4, all results come from one released PO/BT policy (seed 1234). Several runs of the chosen recipe were trained during development and the released one is the best of them by greedy-decoded `train` performance; because that selection, and the within-run checkpoint selection, use only the `train` split, `test` and `ood` play no role in either and the held-out evaluations are not optimistically biased by the choice. Selecting on training reward rather than a held-out criterion is a weaker rule that, if anything, risks favoring an over-fit run, so we make no claim the selected policy is optimal. It also means the reported policy sits at the favourable end of the runs we trained. Note that the four seeds $\{1234, 11, 22, 33\}$ appearing in Table 3 and Section 6.3 are SetPredictor *probe* seeds, not policy seeds; the two axes are reported separately throughout. Policy-seed variance is characterized in Section 6.4, where three further policies trained from scratch under the same recipe show that both load-bearing results — the probe closing the coverage tail, and the encoder's matched-budget advantage — hold on all four runs. What remains uncharacterized is narrower: we did not sweep the full threshold range out of distribution for the additional policies, so the matched-budget comparison on `ood` and `ood-large` rests on the released policy alone, and the no-seed and coordinates-only arms exist only for it.
6. **LSTM, not Transformer, policy.** We trialed a Transformer encoder–decoder policy of roughly $6\times$ the parameters; it reached a comparable coverage–cost operating point (greedy coverage $\approx 0.97$ at $|S|/\mathrm{OPT} \approx 1.15$ versus the LSTM pointer's $\approx 1.21$, sparser but no better on the trade-off), one training run failed to converge, and we found no improvement justifying the added capacity. We read this as overparametrization relative to the scalar RL signal rather than evidence against attention models in general, consistent with the fact that the probe, a small Transformer trained on dense per-vertex labels, does work; a careful attention-model study is future work.
7. **Decode-time search and stronger decoders.** We tested decode-time search of the policy seed by best-of-$K$ stochastic sampling with length-normalized scoring (Section 6.5, Table 12): geo-free likelihood-ranked search matched greedy and did not close the tail, so the residual tail is not a decoding artifact. We did not implement exhaustive beam search (the decoder loop is not exposed for per-step beam expansion, and best-of-$K$ sampling is the stronger search in practice), nor symmetry-exploiting RL such as POMO [16]; these act on the decoder and are orthogonal to the representational question of what the encoder has learned, so we leave them to future work.
8. **Discrete visibility approximation in training.** Both the PO/BT coverage reward and the SetPredictor training targets use discrete visibility sampling at $M = 500$ (Section 4.1). Exact coverage-area computation is used only at evaluation; the per-vertex visibility polygons are computed with CGAL in both training and evaluation. We did not sweep $M$, but

we did measure what the approximation costs the training signal, since PO/BT consumes the *ordering* of two rollouts rather than their reward magnitudes. Re-scoring 1600 rollouts of the frozen policy under both estimators, the discretized coverage is unbiased against exact CGAL (mean difference +0.0003, optimistic on 52% of rollouts) with no trend in the polygon size or the guard count, and the induced pairwise preferences agree on 91.9% of rollout pairs. The disagreements are confined to near-ties: agreement is 100% on pairs whose exact reward gap exceeds 0.10 and falls to chance only as that gap approaches zero, so the flipped pairs are those carrying almost no preference signal in either direction. One asymmetry does survive: because most rollouts sit just below the coverage gate, symmetric sampling noise crosses it upward more often than downward (94 rollouts credited with clearing $\tau$ that exact scoring places below it, against 22 in the reverse direction), which mildly rewards under-covering and may contribute to the residual tail. A larger $M$, or stratified sampling near reflex vertices where the uncovered slivers are hardest to detect, would shrink that effect; quantifying the full sensitivity is left to future work.

One of our replication runs shows how far that asymmetry can be pushed, and we report it because it is a measured instance of this limitation rather than a hypothetical one. Policy seed 22 discovered that the $\tau = 0.99$ gate can be satisfied on the sampled points using a very small guard set: on the training subset it reached an estimated coverage above 0.99 while selecting under 2% of each polygon's vertices, which under Eq. (3) earns a *higher* reward than either of the other seeds' well-behaved endpoints, since the cardinality penalty all but vanishes. This is not an optimization failure — the run found a genuinely better optimum of the training objective — but a demonstration that a 500-sample estimate of coverage is exploitable, in the same way and for the same reason that the local-search oracle can finish below OPT (footnote to Table 3). It also shows why the within-run checkpoint criterion matters: scored on raw coverage alone, selection for that seed lands on epoch 2, the most degenerate point of the run, whereas penalizing guard count recovers a checkpoint that behaves normally under exact scoring on held-out data (Table 11). We take the episode as evidence that the discretized reward, not the optimizer, is the component most in need of strengthening.

9. **Reward-hyperparameter sensitivity.** The reward weights and PO settings ($\lambda = 1.0$, $\rho = 3.0$, $R = 8$ rollouts, $\alpha = 0.05$; Section 5.2) were fixed to the values that produced the released checkpoint rather than swept. The design turns on the *form* of the reward (capping coverage at $\tau$ and linearly penalizing the deficit, Section 4.1) rather than on the precise scalars; a sensitivity study over them would require re-running PO/BT training and is left to future work.
10. **Extreme OOD: the encoder signal is confined to the worst case.** On the `ood-large` split (Table 10, Section 6.3) the frozen-encoder probe lifts worst-case coverage on every probe seed while the no-encoder ablation

cannot, and it leaves the shorter tail on each of the independently trained policies that has a tail to close (Table 11). But the gate-clearing count varies enough across probe seeds that we do not read a gate advantage from this split, and the guard saving seen on the other two splits disappears here. We therefore treat this as evidence of the encoder signal persisting at eleven times the training size rather than as a coverage guarantee at that scale. Note also that the classical greedy and local-search references were not computed for this split, and the vertex-guard optimum is unavailable for its largest polygons, so the guard cost here can be compared only against the policy's own seed.

11. **Training-curve reconstruction.** Figure 2 is reconstructed from four saved checkpoints (epochs 110, 114, 160, 200), not from an online metric log; per-epoch coverage was not preserved during the original PO/BT run. The trace shows only late-training dynamics, which is sufficient to confirm that the gradient direction does not collapse but does not characterize early-training behavior.
12. **The pipeline is *not* invariant to rotation or reflection; vertex-order dependence is removable.** The recurrent encoder reads vertices in file order, so translation and axis-aligned scaling are handled by construction (the input is min–max normalized; Section 5.2), but nothing enforces the other symmetries — and measurement shows they are not acquired. Re-running the frozen policy and probe ($t = 0.20$) on transformed `test` polygons, re-decoding the seed each time and re-normalizing so the model sees a valid input, the gate-clearing count falls from 345/362 under identity to 312 under a 45° rotation, 282 under 90°, 223 under 180°, 205 under mirroring, and 279 under a cyclic re-index; mean coverage falls from 0.9915 to between 0.934 and 0.978, and the worst polygon from 0.857 to as low as 0.382. The rotations are clean tests of congruence: because the input is already normalized to the unit square, a rotation yields a square bounding box and the re-normalization is isotropic, so the transformed polygon is congruent to the original and only its presentation to the encoder changes. The mirroring row should be read more cautiously, since obtaining a validly oriented reflected polygon also reverses the vertex sequence; its drop therefore combines reflection with a full re-indexing and is not attributable to reflection alone. The effect is also not a property of the probe: decoding the policy's seed by itself, with no probe in the path, degrades under the same rotations, so the orientation dependence sits in the encoder. We report this as a genuine limitation on the generality of the learned representation: the encoder's geometry is tied to the orientation and traversal order of the instance ecosystem it was trained on, so the probing result should be read as a statement about representations learned under that convention rather than about orientation-independent geometry.

    The two failure modes are not equally serious, however, because one of them is removable at no cost. Vertex re-indexing can be eliminated outright by canonicalizing the traversal before the encoder reads it: rolling each polygon so that a geometrically determined vertex comes first makes any cyclic re-index exactly invariant by construction, since the roll is undone before the model sees the input. Which vertex to use is a choice, and it matters — so we made it on the disjoint `dev` split, scoring six candidate rules at matched guard budget with the reporting split untouched. The selected rule (the lexicographic maximum in $(x, y)$) then matches native file order on `test`, 346/362 against 345/362 at matched budget, while making the output identical under every cyclic roll of all 362 polygons. Our own first guess, the lexicographic *minimum*, was the worst of the six on `dev` by a wide margin — and catastrophic at a fixed threshold, where it selects far fewer guards than the native order and loses two-thirds of the gate-clearing count. That the same construction can be free or ruinous depending on which vertex is chosen is why the rule has to be selected rather than assumed. Rotation is not helped by the same device, because the canonical vertex is not itself rotation-equivariant; a rotated polygon simply canonicalizes to a different vertex.

Orientation dependence therefore remains the substantive open item, and the natural remedies are architectural: inputs that are invariant by construction (per-vertex edge lengths and turning angles determine a polygon up to rigid motion, and visibility is a rigid-motion invariant, so nothing relevant is lost), a rotation-equivariant encoder, or orientation augmentation during PO/BT training. We regard that as the most promising direction for making the representation more general, and note that the present measurements are what make it a testable hypothesis rather than a guess.

## 9. Conclusion

We asked what a neural policy internalizes about vertex-guard placement when trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference, and we answered it by reading the encoder rather than the decoder: a single-shot probe over the frozen embeddings shortens the in-distribution coverage tail several-fold and compresses the out-of-distribution one by roughly an order of magnitude across four probe seeds, which we take as evidence that the reinforcement-trained encoder had captured more of the placement structure than its decoder emitted as guards, on all four independently trained policies we tested. That structure is tied to the presentation it was trained on — it does not survive rotation or reflection — so the claim is about what this training setup encodes, not about an orientation-independent representation of visibility. The result is a measurement in a deliberately constrained setting, not a competitor to classical solvers on guard count: the learner is not the best AGP heuristic, but it arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about at inference. Three extensions follow naturally. The most promising, given the orientation dependence above, is to make the encoder invariant to rigid motions — by feeding edge lengths and turning angles instead of raw coordinates, by a rotation-equivariant architecture, or simply by orientation augmentation during PO/BT training — which would test whether the structure we measure becomes invariant once the architecture stops fighting it. The companion vertex-order dependence needs no such change: canonicalizing the traversal removes it exactly and at no measured cost (Section 8). Beyond that, a stronger encoder such as a Transformer trained under the same PO/BT objective may shorten the residual tail directly and leave less work for the probe; and a per-instance threshold, predicted geo-free from the same embeddings but trained against coverage-selected targets exactly as the probe is trained against the visibility-derived labels $S^{\star}_{\mathrm{LS}}$, would replace the global knob with an adaptive one, keeping visibility a training-time target, never an inference-time input. More broadly, the encoder-probing decomposition need not be specific to vertex guarding: we conjecture it transfers to other geometric set-cover problems whose coverage oracle is costly at inference (terrain guarding or range-limited watchtower placement, for instance), where geo-free inference stays meaningful precisely because the visibility or coverage oracle one would otherwise consult is expensive to evaluate. Whether a frozen encoder carries the requisite structure in those settings is an open question the present method is designed to ask, and one this study cannot answer on its own: our conclusions concern the PO/BT pointer architecture, replicated across four training runs of it, and establishing that they hold across architectures and learning rules would require probing more than one kind of solver.

### Data and Code Availability

The AGPVG instance library is publicly available [18]. The 313 supplemental `random`-class polygons with their ILP-computed vertex-guard optima, the dataset splits, the trained checkpoints, and all training and evaluation code are available at `https://github.com/dseverdi/AGNet`.

### Conflict of Interest

The authors declare no competing interests.

### References


[1] D. T. Lee, A. K. Lin, Computational complexity of art gallery problems, IEEE Transactions on Information Theory 32 (2) (1986) 276–282.

[2] J. O'Rourke, Art Gallery Theorems and Algorithms, Oxford University Press, 1987.

[3] A. Elnagar, L. Lulu, An art gallery-based approach to autonomous robot motion planning in global environments, in: 2005 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2005, pp. 2079–2084. doi:10.1109/IROS.2005.1545170.

[4] B. S. Bigham, S. Dolatikalan, A. Khastan, Minimum landmarks for robot localization in orthogonal environments, Evol. Intell. 15 (3) (2022) 2235–2238. doi:10.1007/s12065-021-00616-8.
URL https://doi.org/10.1007/s12065-021-00616-8

[5] M. Pan, G. Lin, Y.-W. Luo, B. Zhu, Z. Dai, L. Sun, C. Yuan, Preference optimization for combinatorial optimization problems, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, arXiv:2505.08735.

[6] R. A. Bradley, M. E. Terry, Rank analysis of incomplete block designs: I. the method of paired comparisons, Biometrika 39 (3–4) (1952) 324–345.

[7] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Machine Learning 8 (3-4) (1992) 229–256.

[8] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio, Neural combinatorial optimization with reinforcement learning, arXiv preprint arXiv:1611.09940 (2016).

[9] G. Alain, Y. Bengio, Understanding intermediate layers using linear classifier probes (2017). arXiv:1610.01644.

[10] J. Hewitt, P. Liang, Designing and interpreting probes with control tasks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 2733–2743.

[11] Y. Belinkov, Probing classifiers: Promises, shortcomings, and advances, Computational Linguistics 48 (1) (2022) 207–219.

[12] Z. Zhang, Y. Ma, Z. Cao, H. C. Lau, Probing neural combinatorial optimization models, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 38, 2025, spotlight; arXiv:2510.22131; extended version at OpenReview forum:agEy9hliY1, “Unveiling Neural Combinatorial Optimization Model Representations through Probing”.

[13] R. Narad, L. Boussioux, M. Wagner, Probing neural TSP representations for prescriptive decision support (2026). arXiv:2602.07216.

[14] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 28, 2015.

[15] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: International Conference on Learning Representations (ICLR), 2019.

[16] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, S. Min, POMO: Policy optimization with multiple optima for reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020.

[17] M. C. Couto, P. J. de Rezende, C. C. de Souza, An exact algorithm for minimizing vertex guards on art galleries, International Transactions in Operational Research 18 (4) (2011) 425–448.

[18] M. C. Couto, P. J. de Rezende, C. C. de Souza, Instances for the Art Gallery Problem, `http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery` (2009).

[19] S. K. Ghosh, Approximation algorithms for art gallery problems in polygons, Discrete Applied Mathematics 158 (6) (2010) 718–722.

[20] H. González-Baños, A randomized art-gallery algorithm for sensor placement, Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG) (2001) 232–240.

[21] G. L. Nemhauser, L. A. Wolsey, M. L. Fisher, An analysis of approximations for maximizing submodular set functions—i, Mathematical Programming 14 (1) (1978) 265–294.

[22] B. Ben-Moshe, M. J. Katz, J. S. B. Mitchell, A constant-factor approximation algorithm for optimal 1.5D terrain guarding, SIAM Journal on Computing 36 (6) (2007) 1631–1647.

[23] K. Elbassioni, D. Matijević, J. Mestre, D. Ševerdija, Improved approximations for guarding 1.5-dimensional terrains, CoRR abs/0809.0159v1 (2008).

[24] Y.-H. Liao, P.-C. Chang, H.-H. Li, Reinforcement learning for optimize coverage in art gallery problem using Q-learning based in grid world, IEEE Access 13 (2025) 52711–52724.

[25] M. D. Kaba, M. G. Uzunbas, S. N. Lim, A reinforcement learning approach to the view planning problem (2016). `arXiv:1610.06204`.

[26] M. Deudon, P. Cournut, A. Lacoste, Y. Adulyasak, L.-M. Rousseau, Learning heuristics for the TSP by policy gradient, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR), Vol. 10848 of Lecture Notes in Computer Science, Springer, 2018, pp. 170–181.

[27] Y. Jin, Y. Ding, X. Pan, K. He, L. Zhao, T. Qin, L. Song, J. Bian, Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem, aAAI 2023 (2023). `arXiv:2304.09407`.

[28] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421. `doi:https://doi.org/10.1016/j.ejor.2020.07.063`.
URL `https://www.sciencedirect.com/science/article/pii/S0377221720306895`

[29] I. Mehta, S. Taghipour, S. Saeedi, Pareto frontier approximation network (PA-Net) to solve bi-objective TSP (2022). arXiv:2203.01298.

[30] K. Li, T. Zhang, R. Wang, Deep reinforcement learning for multiobjective optimization, IEEE Transactions on Cybernetics 51 (6) (2021) 3103–3114.

[31] C. Caramanis, D. Fotakis, A. Kalavasis, V. Kontonis, C. Tzamos, Optimizing solution-samplers for combinatorial problems: The landscape of policy-gradient methods, neurIPS 2023 (2023). arXiv:2310.05309.

[32] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011, pp. 627–635.

[33] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780.

[34] The CGAL Project, CGAL user and reference manual, https://doc.cgal.org/5.6/Manual/packages.html (2024).

[35] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.